



VNiVERSiDAD
D SALAMANCA

Facultad de Ciencias
Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

Construcción de una IA para Riichi Mahjong mediante Transformers

Autor

Julia Ruiperez de Brito

Tutor

Prof. Vidal Moreno Rodilla

Departamento

Informática y Automática

Salamanca
Junio 2026

Abstract

This project presents an Artificial Intelligence system capable of selecting discard actions in Riichi Mahjong, a traditional Japanese game of imperfect information.

Using a Transformer-based architecture, the model is trained through imitation learning on a dataset composed of real games collected from Tenhou.net. The model is evaluated in terms of predictive accuracy, and the implications of the obtained results are discussed.

In addition to conventional accuracy metrics, a novel evaluation methodology, referred to as *Strategic Accuracy*, is proposed. This metric is based on semantic categories derived from Mahjong strategy and provides a deeper understanding of the quality of the model's decisions.

The experimental results show that the model achieves a strategic accuracy of approximately 96%, suggesting that it is capable of learning complex gameplay patterns and making informed decisions in situations characterized by imperfect information.

Keywords:

Riichi Mahjong, Artificial Intelligence, Transformers, Imitation Learning, Strategic Accuracy, Model Evaluation, Imperfect Information Games.

Resumen

Este trabajo presenta un sistema de Inteligencia Artificial capaz de tomar decisiones de descarte en Riichi Mahjong, un juego tradicional japonés de información imperfecta caracterizado por un elevado nivel de complejidad estratégica.

Para ello, se desarrolla un modelo basado en la arquitectura Transformer entrenado mediante aprendizaje por imitación a partir de un conjunto de datos compuesto por partidas reales obtenidas de la plataforma Tenhou.net. El rendimiento del modelo se evalúa utilizando métricas convencionales de precisión, analizando además las implicaciones de los resultados obtenidos.

Con el objetivo de complementar estas métricas tradicionales, se propone una nueva metodología de evaluación denominada *Precisión Estratégica*, basada en categorías semánticas propias del juego. Este enfoque permite analizar la calidad de las decisiones generadas por el modelo desde una perspectiva más cercana al razonamiento estratégico empleado por jugadores expertos.

Los resultados experimentales muestran que el modelo alcanza una precisión estratégica cercana al 96%, lo que sugiere que es capaz de aprender patrones de juego complejos y generar decisiones coherentes en entornos caracterizados por información imperfecta.

Palabras clave:

Riichi Mahjong, Inteligencia Artificial, Transformers, Aprendizaje por Imitación, Precisión Estratégica, Evaluación de Modelos, Juegos de Información Imperfecta.

Índice general

1	Introducción	7
2	Objetivos	7
3	Conceptos Teóricos	8
3.1	Reglas Básicas	8
3.2	Heurísticas	11
3.2.1	Ukeire	11
3.2.2	Shanten	11
3.3	Arquitectura Transformer	11
3.3.1	Tokens	13
3.3.2	Embeddings	14
3.3.3	Aprendizaje por Imitación	14
4	Estado del Arte	14
4.1	Suphx	14
4.2	Transformers aplicados al Mahjong	15
4.3	Posicionamiento del presente trabajo	15
5	Herramientas y Entorno de Desarrollo	15
6	Desarrollo del Proyecto	16
6.1	Análisis de Exploración de Datos	16
6.2	Distribución de Acciones	17
6.3	Distribución de la Suma de Acciones Válidas de una Sample	18
6.4	Frecuencia de Piezas Descartadas	19
6.5	Representación de un Estado	19
7	Arquitectura del Modelo	21
7.1	Reconstrucción del estado de juego	22
7.2	Tokenización del estado	23
7.3	Capa de embeddings	23
7.4	Transformer Encoder	23
7.5	Cabeza de política y acciones legales	23
7.6	Justificación de las decisiones arquitectónicas	24
7.7	Elecciones de Diseño de Tokenización	24
7.7.1	Tokens de Mano	25
7.7.2	Tokens de Descarte	25
7.7.3	Tokens de Melds	25
7.7.4	Tokens de Acciones	25
7.7.5	Tokens de Estado de Juego	26
7.7.6	Tokens de Estado de Jugadores	26
7.8	Diseño del Embedding	27
7.8.1	Embedding de Tokens de Piezas	27
7.8.2	Embedding del índice de copia	27
7.8.3	Embedding de Jugador	27
7.8.4	Embeddings Posicionales	27

7.8.5	Embeddings de Metadatos	28
7.8.6	Embedding de Token de Mano	28
7.8.7	Embedding de Token de Descarte	28
7.8.8	Embedding de Token de Meld	29
7.8.9	Embedding de Token de Acción	29
7.8.10	Embedding de Token de Estado de Juego	30
7.8.11	Embedding de Token de Estado de Jugador	30
8	Resultados	31
8.1	Primer Modelo	31
8.2	Segundo Modelo	33
8.3	Validación Estratégica	35
8.3.1	Validación Estratégica del Primer Modelo	35
8.3.2	Validación Estratégica del Segundo Modelo	37
8.3.3	Análisis Comparativo: Impacto del Volumen de Datos	39
9	Conclusiones y Líneas de Trabajo Futuro	41
10	Líneas de Trabajo Futuro	42

Índice de figuras

1	Posición relativa de los jugadores y vientos en una mesa de Riichi Mahjong.	9
2	Arquitectura Estándar de un Transformer	12
3	Arquitectura BERT (izquierda) vs Arquitectura GPT	13
4	Distribución de Número de Datos del Dataset por tipo de Acciones Válidas	17
5	Distribución del número de acciones válidas del schema de Descarte	18
6	Frecuencia de las Piezas Descartadas	19
7	Flujo de procesamiento y arquitectura del modelo propuesto.	22
8	Curvas de pérdida y precisión del primer modelo con 100k de datos de entrenamiento.	31
9	Curvas de pérdida y precisión del primer modelo con 100k de datos de entrenamiento.	32
10	Curvas de precisión top-k del primer modelo con 100k de datos de entrenamiento.	32
11	Distribución de confianza del primer modelo con 100k de datos de entrenamiento.	33
12	Curvas de pérdida y precisión del segundo modelo con 500k de datos de entrenamiento.	34
13	Distribución de confianza del segundo modelo con 500k de datos de entrenamiento.	34
14	Porcentaje de Cada Categoría Estratégica en el Primer Modelo	35
15	Comparación validación de accuracy exacta vs accuracy estratégica en el Primer Modelo	36
16	Distribución de confianza del primer modelo con ajustado al dataset sin opciones unitarias	36
17	Distribución de confianza del primer modelo por categoría de la validación estratégica	37
18	Porcentaje de Cada Categoría Estratégica en el Segundo Modelo	37
19	Comparación validación de accuracy exacta vs accuracy estratégica en el Segundo Modelo	38
20	Distribución de confianza del segundo modelo	38
21	Distribución de confianza del segundo modelo por categoría de la validación estratégica	39
22	Comparación de Accuracy Exacta y Estratégica entre modelos con 100k y 500k de datos	40
23	Comparación de la distribución de categorías estratégicas entre los dos modelos	40

Índice de cuadros

1	Objetivos del proyecto	8
2	Categorías de piezas de Riichi Mahjong	9
3	Formas de ganar en Riichi Mahjong	10
4	Acciones posibles en cada turno de Riichi Mahjong	11
5	Tecnologías empleadas durante el desarrollo del proyecto.	16
6	Schemas del Dataset	17
7	Acciones válidas según melds abiertos	18
8	Campos de un Estado de Mahjong	20
9	Campos de un Jugador en un Estado de Mahjong	21
10	Campos de un Meld en un Estado de Mahjong	21
11	Campos de una Acción Válida en un Estado de Mahjong	21
12	Categorías de Informaciones	24
13	Campos de Token de Mano	25
14	Campos de Token de Descarte	25
15	Campos de Token de Melds	26
16	Campos de Token de Acciones	26
17	Campos de Token de Estado de Juego	26
18	Campos de Token de Estado de Jugadores	27

1 Introducción

Riichi Mahjong es un juego de información imperfecta, caracterizado por una elevada complejidad estratégica e información oculta, que llega a 10^{48} [7] posibles siguientes estados. Además de eso, también existe un gran caos secuencial, debido a que cada jugador tiene la posibilidad de saltar turnos a otros jugadores mediante acciones como el *pon*, el *chi* o el *kan*, lo que incrementa aún más la complejidad del juego.

Este proyecto crea una Inteligencia Artificial que selecciona el mejor descarte dado un estado de juego completo, donde existe información temporal secuencial y estática. El modelo es basado en la arquitectura de Transformers, en la cual se entrena mediante aprendizaje por imitación de 500 mil estados de juego reales obtenidos de la plataforma Tenhou.net, siendo estos juegos de alto nivel, con jugadores profesionales y amateurs avanzados.

Durante el desarrollo del proyecto, obtenemos diferentes métricas de precisión, analizando la precisión global de imitación, así como la precisión estratégica, que se basa en categorías semánticas propias del juego, como *Shanten* y *Ukeire*.

El esquema general de esta memoria será empezará con los objetivos del proyecto, seguido de una sección de teoría, donde se explicará el juego de Mahjong y las heurísticas del juego, así como la arquitectura de Transformers y el aprendizaje por imitación. Después, se hará un repaso del estado del arte, donde se analizarán los trabajos relacionados con el juego de Mahjong y con el uso de Transformers en juegos de información imperfecta. A continuación, se describirán las herramientas utilizadas para el desarrollo del proyecto, y se explicará el proceso de desarrollo del proyecto, desde la preparación de los datos hasta el entrenamiento del modelo y la evaluación de los resultados. Finalmente, se presentarán los resultados obtenidos y se discutirán las conclusiones del proyecto.

Este trabajo se desarrollo utilizando metodología ágil y se documenta de forma detallada cada una de las etapas del proyecto, desde la definición de los objetivos hasta la presentación de los resultados obtenidos, proporcionando una visión completa del proceso de construcción de la IA de Mahjong mediante Transformers.

2 Objetivos

El objetivo principal de este trabajo es conseguir desarrollar una IA de Mahjong mediante la tecnología moderna de Transformers, en contraste con los enfoques de otras IAs como Suphx [7] basado en Redes Convolucionales. Presentando los beneficios y limitaciones de el uso de los Transformers para juegos de información imperfecta.

Nos planteamos los siguientes objetivos:

Cuadro 1: Objetivos del proyecto

ID	Tipo	Objetivo	Prioridad
OBJ-01	General	Desarrollar un modelo basado en Transformers capaz de predecir el descarte realizado por un jugador experto de Riichi Mahjong.	Alta
OBJ-02	Funcional	Diseñar e implementar un parser que reconstruya el estado completo de una partida a partir del conjunto de datos de Tenhou.	Alta
OBJ-03	Funcional	Diseñar una representación del estado del juego basada en tokens adecuada para una arquitectura Transformer.	Alta
OBJ-04	Funcional	Entrenar el modelo mediante aprendizaje por imitación utilizando partidas reales de Riichi Mahjong.	Alta
OBJ-05	Funcional	Evaluar el rendimiento del modelo mediante métricas de clasificación y métricas estratégicas basadas en Shanten y Ukeire.	Alta
OBJ-06	No funcional	Desarrollar un proceso de entrenamiento reproducible y documentado que facilite la experimentación con diferentes configuraciones del modelo.	Media
OBJ-07	General	Analizar el estado del arte de las arquitecturas Transformer aplicadas al Riichi Mahjong y otros juegos de información imperfecta.	Media

3 Conceptos Teóricos

En esta sección trataremos los conceptos teóricos a empezar por las reglas básicas [6] y heurísticas [1] de Riichi Mahjong, el objeto de este proyecto. Luego, pasaremos a la teoría de IA, aprendizaje por imitación y la arquitectura de *Transformers*.

3.1 Reglas Básicas

En un juego de Riichi Mahjong, contamos con 4 jugadores de los cuales 1 es el *Dealer*, representado por el viento **Este**. Cada jugador recibe 13 piezas, que pueden ser de las siguientes:

Suit	Tiles
Manzu (Caracteres)	
Pinzu (Círculos)	
Souzu (Bambús)	
Honores (Vientos y Dragones)	

Cuadro 2: Categorías de piezas de Riichi Mahjong

Las piezas de caracteres, círculos y bambús se numeran del 1 al 9, y las piezas de honores son, respectivamente, **Este**, **Sur**, **Oeste** y **Norte**; y **Dragon Blanco**, **Dragon Verde** y **Dragon Rojo**. Cada una de estas piezas tienen 4 copias, por lo que en total son 136 piezas.

Inicialmente se decide si el juego durará una ronda de mesa **Este** o una ronda hasta **Sur**, cada ronda de **Este** o de **Sur** tiene 4 juegos, donde la mesa obtiene ese viento, y aleatoriamente se escoge una persona para ser el *Dealer*, que estará representado por la pieza del **Este** y se sentará en esa posición. Cada posición de jugadores está en un viento, como podemos ver en la siguiente imagen:

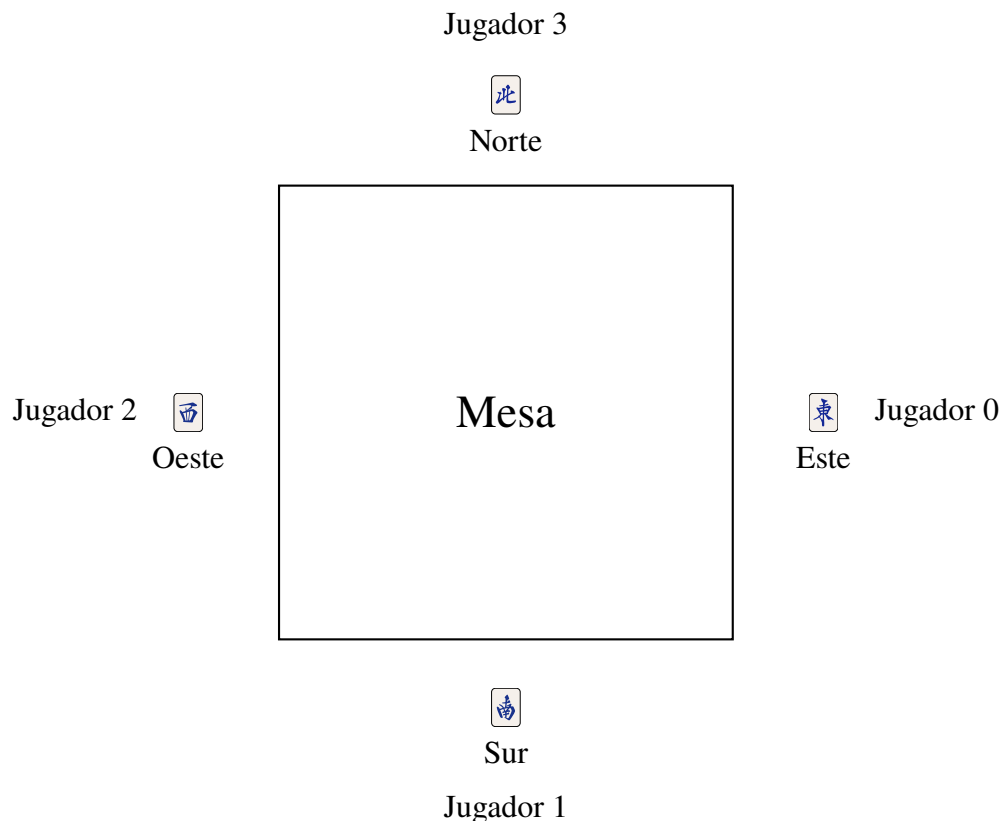


Figura 1: Posición relativa de los jugadores y vientos en una mesa de Riichi Mahjong.

En el caso de que el ganador del juego sea el Este/Dealer, la partida seguirá en el mismo estagio de ronda, por ejemplo, si estamos en el Este 3 y gana el Dealer, se hace más una ronda de Este 3.

El objetivo del juego es ganar las manos mediante *Yaku* (combinaciones de manos que

pueden ganar compuestas de 4 *Melds* y un par), de forma similar al poker. Se puede ver en el Anexo 1 una lista de *Yaku* oficiales.

Cada jugador recibe 13 piezas, y su turno se basa de forma sencilla, en comprar una pieza y descartar otra, hasta que la mano se complete (si posible).

En general, una mano se caracteriza por 4 *Melds* caracterizados por 3 piezas que pueden ser un trio o una secuencia y además de esos 4 *Melds*, un par.

Al momento de completar una mano preparada para ganar (*Tenpai*), el jugador puede ganar cogiendo la pieza de la compra normal, o del descarte de otro jugador. De este modo, los jugadores también deben evitar descartar la pieza que completa la mano de otro jugador, ya que pierde puntos de este modo. En resumen, las formas de ganar son las siguientes:

Forma de Ganar	Descripción
Tsumo	Ganar con la pieza de la compra normal.
Ron	Ganar con la pieza del descarte de otro jugador.

Cuadro 3: Formas de ganar en Riichi Mahjong

El juego también dispone de una pared de piezas, donde se encuentra la llamada **Pared Muerta** (*Dead Wall*) que contiene 14 piezas que nunca se verán dentro del juego, estas 14 piezas son escogidas aleatoriamente al inicio de cada ronda. Además, esta pared muerta contiene lo que llamamos indicadores de *dora*, que son piezas que indican que la siguiente pieza de la secuencia adiciona puntos a la mano ganadora. Para los vientos, el orden de secuencia para indicar el *dora* es Este → Sur → Oeste → Norte → Este, y para los dragones es Rojo → Verde → Blanco → Rojo. Para las piezas de caracteres, círculos y bambús, el orden es 1 → 2 → 3 → ... → 9 → 1.

Durante el juego, un jugador tiene la opción de hacer "llamadas", estas llamadas se resumen a coger el descarte de otro jugador para completar un trio o una secuencia (uno de los *Melds*), abriendo tu mano y bloqueando las piezas que forman el *Meld* llamado, también se puede llamar para completar una cuadrupla, que abrirá otro indicador *dora*, pudiendo aumentar el valor de tu mano o de una mano rival. Sin embargo, hacer llamadas también tiene la contra parte de que tu mano ya no es cerrada, y por lo tanto no podrás declarar *Riichi* como veremos a continuación, una excepción es hacer un *Kan* (cuadrupla) cerrado, esto es, completar una cuadrupla con piezas de tu mano sin haberlas descartado y sin llamar a una pieza del descarte de otro jugador, este movimiento abre las 4 piezas para que los otros jugadores las vean, no obstante, si no se coge del descarte de otro jugador, tu mano sigue cerrada, teniendo la posibilidad aún de hacer *Riichi*.

La regla más importante del Riichi Mahjong: hacer ***Riichi***: Cuando un jugador está en *Tenpai* y tiene la mano cerrada, tiene la opción de declarar *Riichi*, una apuesta de 1000 puntos a que ganará la mano. Esto bloquea la mano del jugador y solo podrá comprar y descartar piezas sin poder modificar más su mano, siendo la única opción posible hacer *Tsumo* o *Ron3*.

Por lo tanto, podemos ver en la siguiente tabla un resumen de las acciones posibles en cada turno:

Acción	Descripción
Comprar	Tomar una pieza de la pared.
Descartar	Descartar una pieza de la mano.
Declarar <i>Riichi</i>	Apostar 1000 puntos a que ganará la mano.
<i>Pon</i>	Coger el descarte de otro jugador para completar un trio.
<i>Chi</i>	Coger el descarte de otro jugador para completar una secuencia.
<i>Kan</i>	Completar una cuadrupla (puede ser de tu mano sin abrir).

Cuadro 4: Acciones posibles en cada turno de Riichi Mahjong

Una última regla importante es el concepto de *furiten*, un jugador no puede ganar mediante *Ron* con una pieza que ha descartado previamente en el juego, esta regla permite el juego defensivo, ya que se sabe a todo momento con cuales piezas otros jugadores **no** pueden ganar.

Con esto finalizamos la explicación de las reglas básicas del juego, y a continuación se explicarán los conceptos de estrategia y teoría que se utilizan para jugar el juego de forma óptima.

3.2 Heurísticas

En esta subsección se definirá dos heurísticas importantes del Mahjong, el *Ukeire* y el *Shanten*, que serán utilizados más adelante como métricas de evaluación del modelo.

3.2.1 Ukeire

El *Ukeire* nada más es que el número de piezas aceptables en cada turno para mejorar tu meld, por ejemplo, para completar un **Meld** con las dos piezas , se pueden aceptar las siguientes piezas:



si ninguna de estas piezas han salido en el juego, entonces tenemos un *Ukeire* de $4 + 3 + 4 + 3 + 4 = 18$. Por lo tanto, queremos maximizar el *Ukeire* en cada turno, ya que esto nos da más posibilidades de mejorar nuestra mano y llegar a *Tenpai*.

3.2.2 Shanten

El *Shanten* se define por el número de turnos que hace mejorar la mano para llegar a *Tenpai*. Un *Shanten* de 0 indica que la mano está en *Tenpai* (falta solo una pieza para ganar), igualmente, una mano en 1-*Shanten* indica que falta una pieza para llegar a *Tenpai* y así por delante. Por lo tanto, queremos minimizar el *Shanten* en cada turno, ya que esto nos acerca a ganar la mano. El *Tenpai* también puede ser definido como el estado de la mano en el que se está a una pieza de ganar, es decir, cuando el *Shanten* es 0.

3.3 Arquitectura Transformer

Los Transformers son una arquitectura de Deep Learning basada en mecanismos de *self-attention*, introducida por Vaswani et al. en el trabajo *Attention Is All You Need*[8].

A diferencia de arquitecturas recurrentes anteriores, los Transformers permiten procesar todos los elementos de una secuencia de forma simultánea, aprendiendo automáticamente las relaciones existentes entre ellos mediante el mecanismo de atención.

La arquitectura original está formada por un codificador (*encoder*) y un decodificador (*decoder*), como se muestra en la Figura 2. Sin embargo, en tareas donde únicamente es necesario comprender una secuencia de entrada, es habitual emplear únicamente el codificador, tal y como hacen modelos como BERT[3] como podemos ver en la Figura 3.

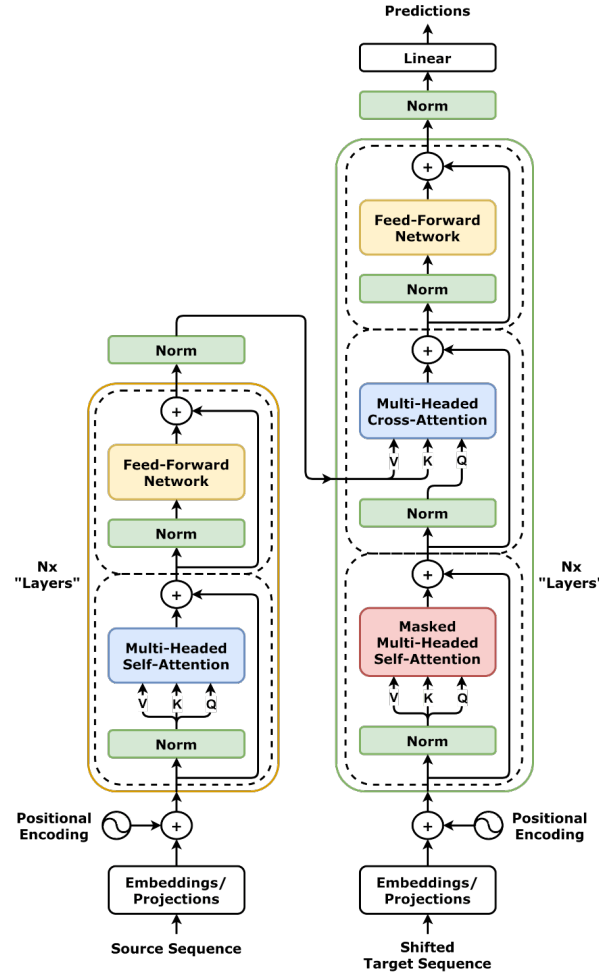


Figura 2: Arquitectura Estándar de un Transformer

Esta aproximación resulta especialmente adecuada para este trabajo, ya que el objetivo no consiste en generar texto, sino en analizar el estado completo de una partida de Riichi Mahjong y predecir la acción de descarte más adecuada. El modelo propuesto recibe como entrada una secuencia de tokens que representan el estado del juego y aprende las relaciones existentes entre ellos para estimar la probabilidad de cada posible acción.

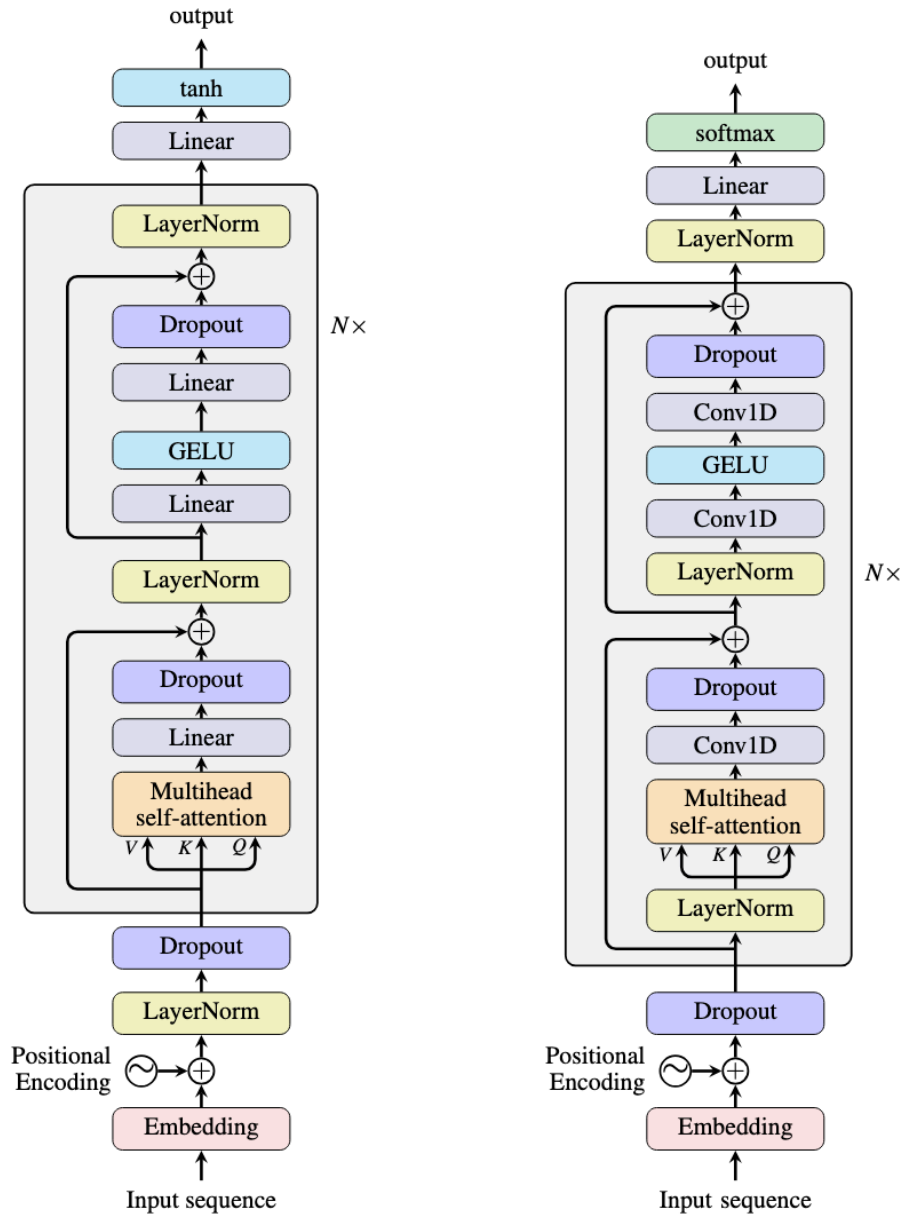


Figura 3: Arquitectura BERT (izquierda) vs Arquitectura GPT

3.3.1 Tokens

Los Transformers no procesan directamente la información original de un problema, sino una secuencia de elementos discretos denominados *tokens*. En procesamiento del lenguaje natural, un token suele corresponder a una palabra o parte de una palabra. Sin embargo, en otros dominios el concepto de token puede adaptarse para representar cualquier unidad elemental de información.

En este proyecto, un token representa un aspecto específico del estado de una partida de Riichi Mahjong, como una ficha de la mano, un descarte, un *meld*, el viento del jugador o el número de fichas restantes. De este modo, el estado completo del juego se transforma en una secuencia estructurada de tokens que puede ser procesada por un Transformer.

3.3.2 Embeddings

Los modelos Transformer requieren que cada token sea representado mediante un vector numérico de dimensión fija antes de ser procesado por la red neuronal. Esta representación continua recibe el nombre de *embedding*.

Durante el entrenamiento, el modelo aprende automáticamente un embedding para cada tipo de token, de forma que aquellos con significado o función similar tienden a situarse próximos entre sí en el espacio vectorial. Esta representación permite capturar relaciones semánticas entre los distintos elementos del estado del juego.

3.3.3 Aprendizaje por Imitación

El aprendizaje por imitación (*Imitation Learning*) consiste en entrenar un modelo para reproducir las decisiones tomadas por un experto a partir de ejemplos previamente observados[5]. En lugar de descubrir una estrategia mediante interacción con el entorno, como ocurre en el aprendizaje por refuerzo, el modelo aprende directamente una función que aproxima el comportamiento del experto.

En el ámbito de los juegos de estrategia, este enfoque permite entrenar modelos utilizando grandes conjuntos de partidas reales, facilitando el aprendizaje de reglas, patrones tácticos y principios estratégicos implícitos en las decisiones de jugadores experimentados.

En este proyecto se emplea aprendizaje por imitación utilizando aproximadamente 500.000 estados de juego obtenidos de partidas reales de alto nivel de la plataforma Tenhou.net. El objetivo del modelo consiste en predecir el descarte realizado por el jugador experto para un estado concreto de la partida.

4 Estado del Arte

El desarrollo de sistemas de Inteligencia Artificial capaces de jugar al Riichi Mahjong ha experimentado un avance significativo durante los últimos años gracias al empleo de técnicas de aprendizaje profundo y aprendizaje por refuerzo. Entre los modelos más representativos destacan Suphx, Mortal y NAGA, que representan diferentes aproximaciones al problema.

4.1 Suphx

Suphx[7], desarrollado por Microsoft Research, constituye uno de los primeros sistemas capaces de alcanzar un nivel profesional en Riichi Mahjong. Su arquitectura combina redes neuronales convolucionales (CNN) con técnicas de aprendizaje por refuerzo, representando el estado de la partida mediante una codificación espacial similar a una imagen.

Gracias a esta aproximación, Suphx consiguió situarse entre el 0.1% de mejores jugadores de la plataforma Tenhou.net, convirtiéndose en un referente dentro de la investigación en Inteligencia Artificial aplicada al Mahjong.

Sin embargo, la representación espacial utilizada implica un elevado coste computacional y requiere un preprocesamiento considerable para transformar el estado del juego en tensores bidimensionales adecuados para una CNN.

4.2 Transformers aplicados al Mahjong

La aparición de la arquitectura Transformer permitió explorar nuevas formas de representar el estado de una partida sin necesidad de convertirlo en una estructura espacial.

Uno de los primeros trabajos en esta dirección fue Tjong[2], que emplea Transformers para representar el estado del juego mediante secuencias de tokens. Esta aproximación reduce significativamente la complejidad del preprocesamiento y facilita la incorporación de nuevas características al modelo. No obstante, Tjong fue diseñado para Mahjong chino, cuyas reglas difieren considerablemente del Riichi Mahjong.

Otro modelo destacado es Mortal, un proyecto de código abierto basado también en Transformers y técnicas de aprendizaje por imitación combinadas con aprendizaje por refuerzo. Mortal ha demostrado un rendimiento competitivo frente a Suphx y constituye actualmente uno de los motores abiertos más potentes para Riichi Mahjong. Sin embargo, la documentación técnica disponible es limitada y gran parte de su implementación se centra en la optimización del rendimiento, dificultando su utilización como referencia académica para comprender el diseño completo del modelo.

Por otra parte, NAGA es un motor comercial ampliamente utilizado por jugadores de Riichi Mahjong para analizar partidas y estudiar estrategias. Aunque ha demostrado un elevado nivel de juego, su arquitectura y proceso de entrenamiento no son públicos, por lo que únicamente puede considerarse como una referencia del estado actual de la tecnología y no como un modelo reproducible desde el punto de vista científico.

4.3 Posicionamiento del presente trabajo

A diferencia de Suphx, el presente trabajo no emplea representaciones espaciales mediante CNN, sino una representación basada en tokens procesada directamente por un Transformer de tipo encoder.

Asimismo, frente a modelos como Mortal o NAGA, cuyo objetivo principal consiste en maximizar el rendimiento competitivo mediante arquitecturas complejas y técnicas avanzadas de entrenamiento, este proyecto persigue un objetivo diferente: desarrollar un modelo reproducible y comprensible desde un punto de vista académico que permita estudiar la aplicación de Transformers al problema del descarte en Riichi Mahjong.

Para ello se emplea aprendizaje por imitación sobre partidas reales de Tenhou.net, utilizando una representación explícita del estado del juego y evaluando el modelo no únicamente mediante precisión de clasificación, sino también mediante métricas estratégicas propias del dominio, como Shanten y Ukeire.

5 Herramientas y Entorno de Desarrollo

El desarrollo del proyecto se ha realizado principalmente utilizando el lenguaje de programación Python, empleando un entorno virtual gestionado mediante Conda y aceleración por GPU mediante CUDA para el entrenamiento de los modelos desarrollados con la biblioteca PyTorch.

La metodología de trabajo seguida ha sido eminentemente iterativa y experimental. En lugar de implementar directamente un modelo final, el proyecto se ha dividido en diferentes fases documentadas mediante cuadernos de Jupyter Notebook, permitiendo registrar de forma independiente cada etapa del desarrollo: exploración inicial del conjunto de datos, diseño de la representación del estado de juego, construcción progresiva de la arquitectura Transformer,

validación mediante pruebas de sobreajuste, entrenamiento a diferentes escalas y análisis estratégico de los resultados obtenidos.

Este enfoque ha facilitado la experimentación continua sobre distintos tamaños de conjunto de datos, configuraciones de entrenamiento e incluso diferentes métricas de evaluación, permitiendo justificar cada decisión adoptada durante el desarrollo del modelo.

Todo el código fuente del proyecto, así como los distintos experimentos realizados, se encuentran versionados mediante Git y alojados en un repositorio público de GitHub:

<https://github.com/Ruipi/tfg>

Los experimentos de entrenamiento se ejecutaron sobre un equipo equipado con una GPU NVIDIA GeForce RTX 5060 Laptop, lo que permitió acelerar significativamente el proceso de entrenamiento del modelo Transformer y realizar múltiples iteraciones experimentales en tiempos razonables.

Elemento	Herramienta
Lenguaje	Python 3.11
Gestión de entorno	Conda (Miniforge)
Deep Learning	PyTorch
GPU	CUDA
Visualización	Matplotlib, Seaborn
Análisis de datos	Pandas, NumPy
Base de datos	SQLite
Desarrollo experimental	Jupyter Notebook
Control de versiones	Git + GitHub
Editor	Visual Studio Code

Cuadro 5: Tecnologías empleadas durante el desarrollo del proyecto.

6 Desarrollo del Proyecto

En esta sección explicaremos como ha sido desarrollado el proyecto, empezando por el Análisis de Exploración de Datos (EDA); el planteamiento de la arquitectura, donde se verán las decisiones tomadas para la construcción del modelo; la construcción del modelo; y las iteraciones de desarrollo.

6.1 Análisis de Exploración de Datos

A partir del Dataset de Kaggle 4-players Riichi Mahjong Dataset se creó un notebook donde se explora todos los *schemas* del dataset.

El dataset es compuesto por 7 *schemas*:

Cuadro 6: Schemas del Dataset

ID	Nombre	Descripción
0	Skip	No realizar ninguna acción; pasar el turno.
1	Discard	Descartar una ficha de la mano.
2	Chi	Robar la ficha descartada por el jugador anterior para formar una secuencia.
3	Pon	Robar la ficha descartada por cualquier jugador para formar un trío.
4	DaiMinKan	Robar la ficha descartada por cualquier jugador para formar un cuádruple abierto.
5	ShouMinKan	Añadir una ficha propia a un trío abierto (Pon) para formar un cuádruple.
6	AnKan	Declarar un cuádruple cerrado con cuatro fichas de la propia mano.
7	Riichi	Declarar <i>Tenpai</i> con la mano cerrada, apostando 1000 puntos.

Y a partir de ellos, se analizaron los siguientes datos.

6.2 Distribución de Acciones

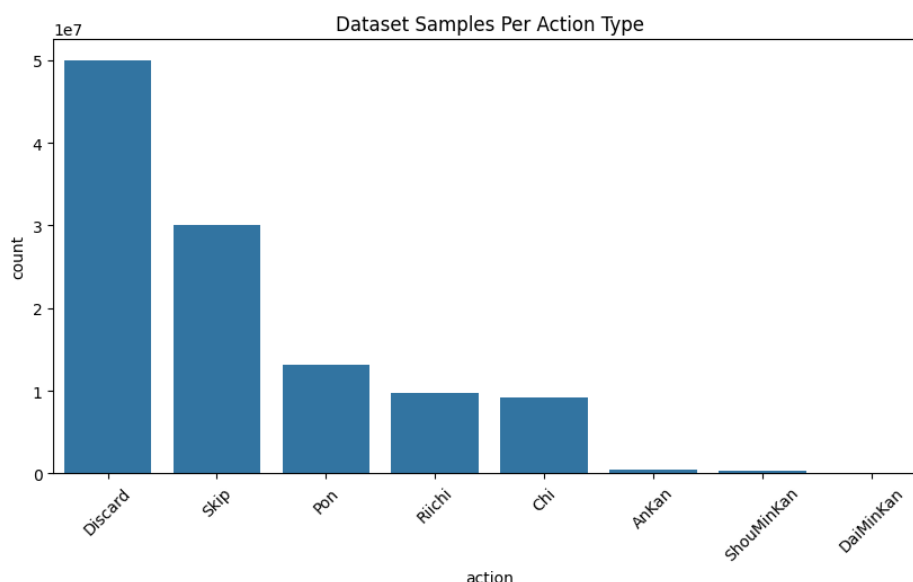


Figura 4: Distribución de Número de Datos del Dataset por tipo de Acciones Válidas

En este gráfico podemos observar un desbalance de acciones, no obstante, se observó que este desbalance es fiel a la realidad, ya que en juegos de alto nivel la cantidad de acciones que abren la mano muy raramente ocurre, y la cantidad de Kan, además de ser baja porque las probabilidades de conseguir las 4 copias de una ficha en el juego es baja, también es un movimiento de mucho riesgo, por lo que eso explica las pocas ocurrencias en la distribución. En conclusión, apesar de este desbalance que se tendrá que llevar en cuenta para los modelos a parte del descarte, en general es correcto que el modelo se entrene sobre estas cantidades inicialmente. Sin embargo, nuestro modelo actualmente solo está entrenado sobre la acción de Descartes, siendo las otras acciones una implementación futura.

6.3 Distribución de la Suma de Acciones Válidas de una Sample

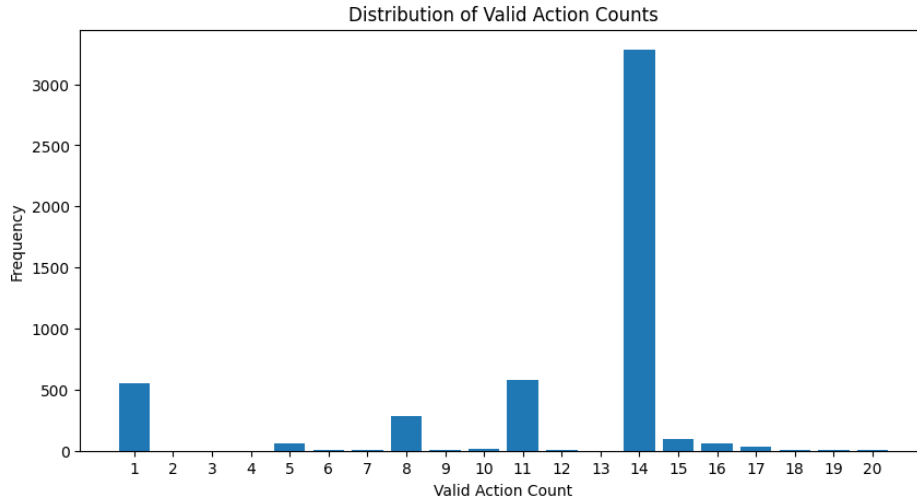


Figura 5: Distribución del número de acciones válidas del schema de Descarte

Observamos que la mayor concentración ocurre en 14 acciones validas, esto ocurre debido a que en la mayor parte del tiempo, el jugador debe descartar una pieza de 14. Luego, aparecen picos en 11, 8 y 5 acciones válidas; esto corresponde naturalmenete a estados de manos abiertas, como podemos ver en la siguiente tabla:

Cuadro 7: Acciones válidas según melds abiertos

Acciones válidas	Melds abiertos
11	1
8	2
5	3
2	4

El 2 obtiene un número bajo debido a que jugadores de alto nivel no suelen abrir la mano hasta que resten 2 piezas de acciones posibles, ya que se busca maximizar las acciones hasta llegar a *Tenpai*.

Además de esto, vemos un alto número donde solamente existe 1 acción válida. Esto se explica con *Riichi*, ya que el *Riichi* bloquea la mano del jugador, permitiendo apenas descartar la pieza comprada o ganar con ella. En función de esto, consideramos limpiar el dataset de estos casos, ya que introduce mucho ruido al modelo, considerando que el jugador ya no tiene autonomía a partir del momento en que hace *Riichi*.

Finalmente, el número de acciones válidas restante se explica debido a posibles acciones de *Riichi* con diferentes piezas.

6.4 Frecuencia de Piezas Descartadas

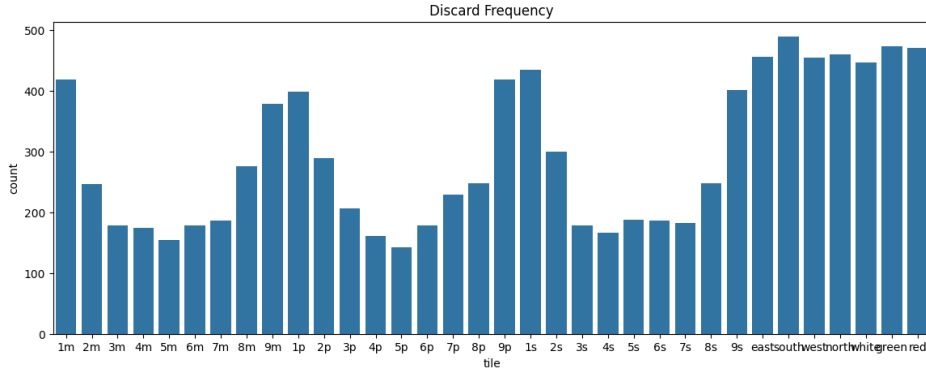


Figura 6: Frecuencia de las Piezas Descartadas

En este gráfico podemos ver que las piezas con mayor frecuencia de dedscarte son los Honores (Vientos y Dragones), ver Tabla 2, y las piezas terminales (unos y nueves), esto se debe a que estas piezas disminuyen el *Ukeire*, ver 3.2.1 Ukeire.

6.5 Representación de un Estado

Esta es la representación de un estado de mahjong dentro del Dataset:

Listing 1: Ejemplo de un estado

```
1 {
2   { 'round_wind': 0,
3   'num_honba': 1,
4   'num_riichi': 0,
5   'player_wind': 0,
6   'position': 0,
7   'remain_tiles': 15,
8   'dora_indicators': [7, 5],
9   'hand_tiles': [1, 1, 2, 3, 3, 12, 13, 18, 19, 22, 22],
10  'players': { '0': { 'points': 41700,
11    'riichi': False,
12    'discards': [28, 27, 9, 16, 10, 6, 15, 20, 13, 32, 32, 0, 33, 33],
13    'melds': [{ 'type': 2, 'tiles': [92, 98, 103, -1], 'who': [0, 0, 3, -1] }],
14    'tsumo_giri': [0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0]},
15  '1': { 'points': 19000,
16    'riichi': False,
17    'discards': [30, 27, 32, 18, 31, 17, 26, 26, 24, 2, 0, 14, 30, 10],
18    'melds': [],
19    'tsumo_giri': [0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0]},
20  '2': { 'points': 20300,
21    'riichi': False,
22    'discards': [18, 17, 11, 16, 24, 25, 28, 23, 33, 32, 20, 7, 30, 17],
```

```

23     'melds': [{ 'type': 5, 'tiles': [124, 126, 127, 125], 'who': [2,
24                 2, 1, 2]}],
25     { 'type': 2, 'tiles': [5, 13, 10, -1], 'who': [2, 2, 1, -1]}],
26     'tsumo_giri': [0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1]},
27     '3': { 'points': 19000,
28           'riichi': False,
29           'discards': [28, 29, 30, 18, 0, 11, 9, 27, 27, 0, 29, 26, 24, 25
30                       ],
31           'melds': [],
32           'tsumo_giri': [0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1]}],
33     'valid_actions': [{ 'type': 1, 'tiles': [1], 'who': [0, -1, -1, -1]
34                       },
35                       { 'type': 1, 'tiles': [1], 'who': [0, -1, -1, -1]}],
36                       { 'type': 1, 'tiles': [2], 'who': [0, -1, -1, -1]}],
37                       { 'type': 1, 'tiles': [3], 'who': [0, -1, -1, -1]}],
38                       { 'type': 1, 'tiles': [3], 'who': [0, -1, -1, -1]}],
39                       { 'type': 1, 'tiles': [12], 'who': [0, -1, -1, -1]}],
40                       { 'type': 1, 'tiles': [13], 'who': [0, -1, -1, -1]}],
41                       { 'type': 1, 'tiles': [18], 'who': [0, -1, -1, -1]}],
42                       { 'type': 1, 'tiles': [19], 'who': [0, -1, -1, -1]}],
43     'action_idx': 7}
44 }

```

Donde podemos obtener las siguientes informaciones:

Cuadro 8: Campos de un Estado de Mahjong

Campo	Tipo	Descripción
round_wind	int	Viento de la ronda (0-3)
num_honba	int	Número de honba (0-4)
num_riichi	int	Número de riichi's (0-3)
player_wind	int	Viento del jugador (0-3)
position	int	Posición del jugador (0-3)
remain_tiles	int	Número de fichas restantes en el muro (0-70)
dora_indicators	list[int]	Lista de indicadores de dora (0-33)
hand_tiles	list[int]	Lista de fichas en la mano del jugador (0-33)
players	dict[int, dict]	Información de los jugadores, incluyendo puntos, riichi, descartes, melds y tsumo_giri.
valid_actions	list[dict]	Lista de acciones válidas para el jugador actual.
action_idx	int	Índice de la acción tomada por el jugador actual.

De estos datos, podemos aún extraer los campos de players:

Cuadro 9: Campos de un Jugador en un Estado de Mahjong

Campo	Tipo	Descripción
points	int	Puntos del jugador (0-50000)
riichi	bool	Indica si el jugador ha declarado riichi (True/False)
discards	list[int]	Lista de fichas descartadas por el jugador (0-33)
melds	list[dict]	Lista de melds del jugador, cada uno con tipo, fichas y quién contribuyó.
tsumo_giri	list[int]	Lista indicando si cada ficha fue descartada inmediatamente después de ser robada (1) o no (0).

Campos de melds:

Cuadro 10: Campos de un Meld en un Estado de Mahjong

Campo	Tipo	Descripción
type	int	Tipo de meld (2: Pon, 3: Chii, 4: DaiMinKan, 5: ShouMinKan, 6: AnKan)
tiles	list[int]	Lista de fichas que componen el meld (0-33)
who	list[int]	Lista indicando quién contribuyó a cada ficha del meld (-1 para fichas propias, 0-3 para otros jugadores)

Y, por fin, campos de valid_actions:

Cuadro 11: Campos de una Acción Válida en un Estado de Mahjong

Campo	Tipo	Descripción
type	int	Tipo de acción (0: Skip, 1: Discard, 2: Chi, 3: Pon, 4: DaiMinKan, 5: ShouMinKan, 6: AnKan, 7: Riichi)
tiles	list[int]	Lista de fichas involucradas en la acción (0-33)
who	list[int]	Lista indicando quién realiza la acción y quién contribuye a ella (-1 para el jugador actual, 0-3 para otros jugadores)

7 Arquitectura del Modelo

Como se explicó en la Subsección 3.3, el modelo desarrollado en este trabajo utiliza una arquitectura basada en el codificador de un Transformer. La elección de una arquitectura de tipo *encoder-only* se debe a que el objetivo del sistema no consiste en generar una secuencia de salida, sino en analizar un estado completo de una partida de Riichi Mahjong y seleccionar una acción de descarte entre las opciones legales disponibles.

La Figura 7 muestra el flujo general de procesamiento, desde la reconstrucción del estado almacenado en el conjunto de datos hasta la predicción final realizada por el modelo.

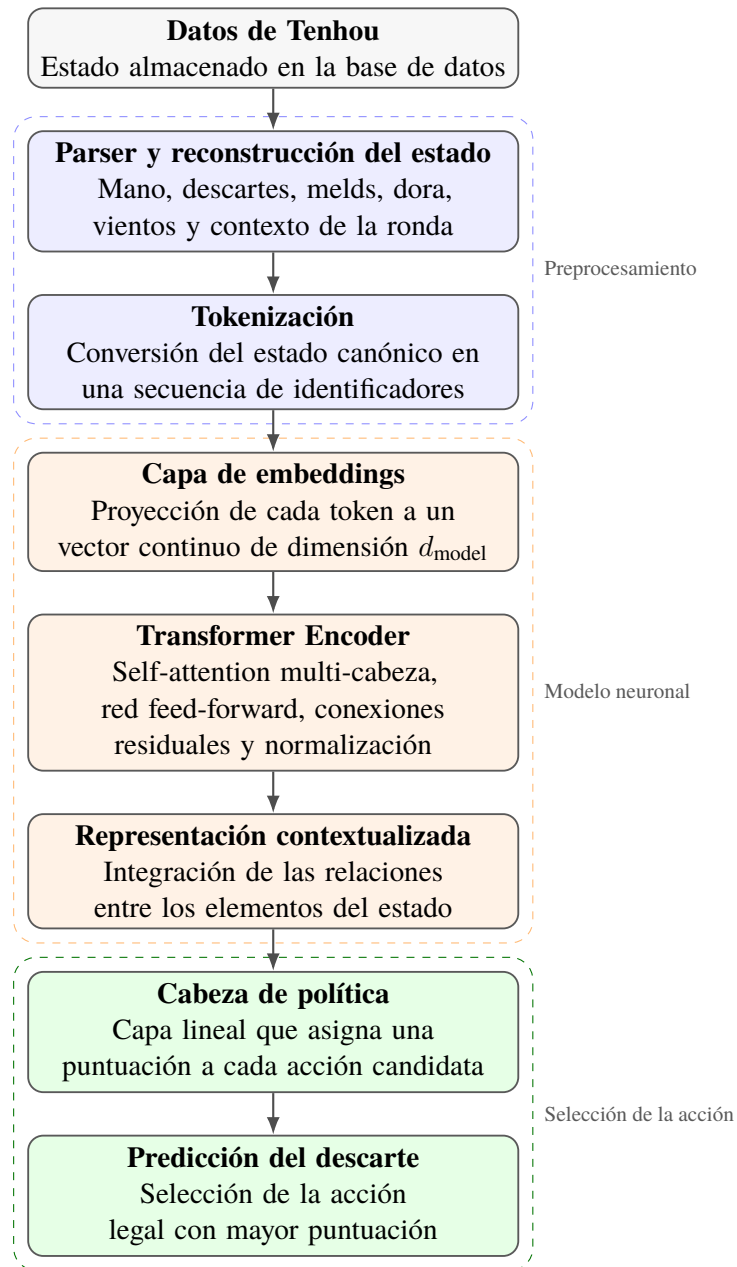


Figura 7: Flujo de procesamiento y arquitectura del modelo propuesto.

7.1 Reconstrucción del estado de juego

Los registros del conjunto de datos no pueden utilizarse directamente como entrada de una red neuronal, ya que contienen la información de la partida en un formato estructurado que debe ser interpretado y normalizado previamente.

Por este motivo, se desarrolló un parser encargado de reconstruir una representación canónica del estado de juego. Esta representación reúne la información considerada relevante para la decisión de descarte, como la mano del jugador, los descartes visibles, las combinaciones abiertas, los indicadores de Dora, los vientos de ronda y de asiento, y otros elementos asociados al contexto de la partida.

7.2 Tokenización del estado

Una vez reconstruido el estado, su información se transforma en una secuencia de tokens mediante un tokenizador desarrollado específicamente para este proyecto.

Cada token representa una unidad semántica del estado de juego y como se verá en la Subsección 7.7 tendremos para cada información del estado sus respectivos tokens. Por ejemplo, pueden emplearse tokens para codificar una ficha presente en la mano, un descarte realizado por un jugador, una combinación abierta, un indicador de Dora o el viento correspondiente al jugador.

Esta representación simbólica se eligió como alternativa a las codificaciones espaciales empleadas por modelos basados en redes convolucionales. En lugar de transformar el estado del Mahjong en una estructura bidimensional similar a una imagen, la tokenización permite representar sus componentes de forma explícita y compacta.

La secuencia resultante mantiene diferenciada la naturaleza de cada elemento del estado y facilita la incorporación de nueva información sin necesidad de rediseñar completamente la estructura de entrada.

7.3 Capa de embeddings

Los identificadores discretos obtenidos durante la tokenización no pueden ser procesados directamente por el Transformer. Por ello, cada token se proyecta a un vector continuo de dimensión fija mediante una capa de embeddings entrenable.

Durante el entrenamiento, el modelo ajusta estas representaciones para capturar relaciones entre los distintos elementos del estado. De esta forma, tokens asociados a fichas, zonas del tablero o situaciones estratégicas relacionadas pueden adquirir representaciones útiles para la predicción.

La salida de esta etapa es una matriz de vectores, donde cada fila corresponde a la representación numérica de uno de los tokens del estado.

7.4 Transformer Encoder

La secuencia de embeddings se introduce posteriormente en un Transformer Encoder. Este bloque está compuesto por varias capas que combinan mecanismos de *multi-head self-attention*, redes neuronales *feed-forward*, conexiones residuales y normalización.

El mecanismo de autoatención permite que cada elemento de la entrada tenga en cuenta la información proporcionada por el resto de tokens. Esta propiedad resulta especialmente útil en Riichi Mahjong, ya que la conveniencia de descartar una ficha no depende únicamente de la composición de la mano, sino también de otros factores, como los descartes visibles, las combinaciones abiertas, los indicadores de Dora o el progreso de la ronda.

Por tanto, la salida del codificador no representa cada token de forma aislada, sino contextualizada respecto al estado completo de la partida.

7.5 Cabeza de política y acciones legales

La representación contextual producida por el Transformer se utiliza para asignar una puntuación a las acciones de descarte disponibles. Esta parte final del modelo recibe el nombre de cabeza de política o *policy head*.

Para cada muestra se construye el conjunto de acciones legales a partir de las fichas que el jugador puede descartar en dicho estado. La cabeza de política calcula un valor o *logit* para

cada una de estas acciones candidatas. Como el número de acciones legales puede variar entre estados, la salida del modelo también puede presentar una longitud diferente para cada muestra.

La acción predicha corresponde al descarte con mayor puntuación:

$$\hat{a} = \arg \max_{a \in A(s)} z_a, \quad (1)$$

donde $A(s)$ representa el conjunto de acciones candidatas para el estado s y z_a el logit asignado por la cabeza de política a la acción a .

Durante el entrenamiento, los logits generados por el modelo se comparan con la acción realizada por el jugador experto mediante una función de pérdida de entropía cruzada (*Cross-Entropy Loss*) [4]. Esta función aplica internamente una normalización Softmax sobre los logits para obtener una distribución de probabilidad y penaliza aquellas predicciones que asignan una baja probabilidad a la acción correcta. Como consecuencia, el modelo aprende progresivamente a incrementar la puntuación de las decisiones observadas en el conjunto de entrenamiento.

7.6 Justificación de las decisiones arquitectónicas

La arquitectura fue diseñada atendiendo tanto a las características del problema como a los recursos disponibles para el desarrollo del proyecto.

En primer lugar, se eligió un Transformer Encoder porque el problema se formula como una tarea de clasificación condicionada por un estado completo. No es necesario generar una secuencia ni predecir estados futuros, por lo que una arquitectura *encoder-decoder* introduciría complejidad adicional sin aportar una ventaja directa para este objetivo.

En segundo lugar, la representación mediante tokens permite mantener una correspondencia clara entre los elementos del estado y la información procesada por el modelo. Esta decisión mejora la interpretabilidad del preprocesamiento y evita la necesidad de construir una representación espacial específica para redes convolucionales.

Finalmente, la predicción se limita a las acciones legales de cada estado. Esta elección evita que el modelo tenga que asignar probabilidad a descartes imposibles y adapta la salida a la naturaleza variable de las decisiones disponibles en una partida de Mahjong.

En conjunto, la arquitectura propuesta busca alcanzar un equilibrio entre capacidad de representación, coste computacional y claridad de implementación.

7.7 Elecciones de Diseño de Tokenización

En primer lugar, necesitamos definir las categorías de información de un estado:

Categoría	Ejemplos	Ordenado	Público	Dinámico
Piezas de mano	5m, este	No	No	Sí
Descartes	descarte 9m	Sí	Sí	Sí
Melds	pon/chii/kan	Parcialmente	Sí	Sí
Acciones candidatas	descartar 5m	No	N/A	Sí
Contexto	Viento de la Ronda	No	Sí	Lento
Estatus de los jugadores	riichi, puntos	No	Sí	Sí

Cuadro 12: Categorías de Informaciones

Un estado de Mahjong será representado como una colección de tokens semánticos. Cada token representará una categoría diferente de información de juego y podrá contener información de metadatos del juego.

A continuación veremos, para cada categoría, cuáles son los campos que tendrá el token.

7.7.1 Tokens de Mano

Piezas de la mano representarán la identidad de la pieza, una distinción de duplicados (cuál copia de la pieza es), y un indicador de si es dora rojo.

Campo	Objetivo
token_type	identifica el token HAND
tile_type	pieza de Mahjong
copy_index	diferencia copias de piezas
is_red	soporte de dora rojo

Cuadro 13: Campos de Token de Mano

7.7.2 Tokens de Descarte

Los tokens de descarte representan eventos de descartes que son públicos y ordenados, los campos de un token de descarte son los siguientes:

Campo	Objetivo
token_type	identifica el token DISCARD
player	jugador que ha descartado la pieza
tile_type	pieza de Mahjong
copy_index	diferencia copias de piezas
is_red	soporte de dora rojo
is_tsumogiri	si la pieza descartada fue una pieza inmediatamente comprada
is_riichi_declaration	si al descartar esa pieza se declara riichi
global_order	orden cronológico del descarte en la ronda
player_discard_order	índice de descarte para el jugador

Cuadro 14: Campos de Token de Descarte

7.7.3 Tokens de Melds

Los Tokens de Melds representan eventos de melds que son públicos y ordenados, los campos de un token de meld son los siguientes:

Message sent Undo 1 of 4,365

7.7.4 Tokens de Acciones

Los Tokens de Acciones representan eventos de acciones candidatas que son públicos y ordenados, los campos de un token de acción son los siguientes:

Campo	Objetivo
token_type	identifica el token MELD
player	jugador que posee el meld
meld_type	chii / pon / kan
tile_types	piezas que componen el meld
copy_indices	distincion de duplicados
red_flags	soporte de dora rojo
source_players	jugadores que contribuyeron para cada pieza
global_order	orden cronologica de llamada

Cuadro 15: Campos de Token de Melds

Campo	Objetivo
token_type	identifica el token ACTION
acting_player	jugador que realiza la acción
action_index	índice dentro del conjunto de candidatos legales
action_type	descartar / riichi / pon / etc
tile_types	piezas involucradas en la acción
copy_indices	distinción de duplicados
red_flags	soporte de dora rojo
source_players	fuelle de las piezas llamadas
global_order	orden cronológico de la acción

Cuadro 16: Campos de Token de Acciones

7.7.5 Tokens de Estado de Juego

Los Tokens de Estado de Juego representan información contextual del juego que es pública y no ordenada, los campos de un token de estado de juego son los siguientes:

Campo	Objetivo
token_type	identifica el token GAME_STATE
round_wind	ronda Este/Sur
honba_count	honba actual
riichi_sticks	depósitos de riichi
remaining_tiles	piezas restantes en la pared
dora_indicators	indicadores de dora actuales

Cuadro 17: Campos de Token de Estado de Juego

7.7.6 Tokens de Estado de Jugadores

Los Tokens de Estado de Jugadores representan información contextual de cada jugador que es pública y no ordenada, los campos de un token de estado de jugadores son los siguientes:

Campo	Objetivo
token_type	identifica el token PLAYER_STATE
player	identidad del jugador
seat_wind	este/sur/oeste/norte
score	puntos actuales
riichi_status	si el jugador ha declarado riichi
placement	posición actual
open_hand	si el jugador tiene mano abierta

Cuadro 18: Campos de Token de Estado de Jugadores

7.8 Diseño del Embedding

La estructura general del embedding es la siguiente:

$$\text{Embedding}(\text{token}) = \text{token_type_embedding} + \text{semantic_embedding} + \text{metadata_embedding} + \text{positional_embedding}$$

Tipos diferentes de tokens utilizarán diferentes embeddings a depender de su estructura semántica.

Cada categoría de token tendrá un embedding dedicado de tipo de token, que podrá ser uno de los siguientes: HAND, DISCARD, MELD, ACTION, GAME_STATE o PLAYER_STATE.

A continuación, veremos la estructura de embedding para cada token semántico.

7.8.1 Embedding de Tokens de Piezas

Este Embedding codifica la identidad de la pieza, por ejemplo: 1m, 5p, east, red_dragon. Los embeddings de piezas son compartidos entre los tokens de mano, descarte, melds y de acciones.

7.8.2 Embedding del índice de copia

Instancias de piezas duplicadas (por ejemplo, dos 5m) se distinguen mediante un índice de copia. Este embedding permite al modelo diferenciar entre estas instancias.

7.8.3 Embedding de Jugador

Este embedding codifica la identidad del jugador, por ejemplo, jugador 0, jugador 1, etc. Los embeddings de jugadores son compartidos entre los tokens de descarte, melds, acciones y estado de jugadores.

7.8.4 Embeddings Posicionales

Los embeddings posicionales preservan el orden temporal de los eventos de juego, como descartes, momento en que se realizaron melds y extensiones de trayectorias de acciones. Existen dos tipos de esquemas posicionales: orden cronológica global y orden de descarte local de jugador.

7.8.5 Embeddings de Metadatos

Los embeddings de metadatos codifican información adicional relevante para cada token, como indicadores de dora, estado de riichi, y otros atributos contextuales. Estos embeddings permiten al modelo capturar relaciones complejas entre los tokens y el contexto del juego.

7.8.6 Embedding de Token de Mano

Los tokens de mano representan piezas escondidas del jugador actual y es definida por la siguiente estructura de embedding:

$$E_{\text{hand}} = E_{\text{token_type}} + E_{\text{tile}} + E_{\text{copy}} + E_{\text{red}}$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo HAND,
- E_{tile} representa la identidad semántica de la pieza,
- E_{copy} distingue entre instancias duplicadas de piezas,
- E_{red} indica si la pieza es un dora rojo.

7.8.7 Embedding de Token de Descarte

Los tokens de descarte representan eventos de descartes que son públicos y ordenados, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_{\text{discard}} = & E_{\text{token_type}} + E_{\text{tile}} + E_{\text{copy}} + \\ & E_{\text{red}} + E_{\text{player}} + E_{\text{tsumogiri}} + \\ & E_{\text{riichi_declaration}} + E_{\text{global_position}} + E_{\text{local_position}} \end{aligned} \quad (2)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo DISCARD,
- E_{tile} representa la identidad semántica de la pieza,
- E_{copy} distingue entre instancias duplicadas de piezas,
- E_{red} indica si la pieza es un dora rojo,
- E_{player} identifica al jugador que ha descartado la pieza,
- $E_{\text{tsumogiri}}$ indica si la pieza descartada fué una pieza inmediatamente comprada,
- $E_{\text{riichi_declaration}}$ indica si al descartar esa pieza se declara riichi,
- $E_{\text{global_position}}$ representa el orden cronológico global del descarte,
- $E_{\text{local_position}}$ representa el orden de descarte relativo al jugador.

7.8.8 Embedding de Token de Meld

Tokens de Melds representan llamadas hechas durante la partida, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_{\text{meld}} = & E_{\text{token_type}} + E_{\text{player}} + E_{\text{meld_type}} + \\ & E_{\text{tile_set}} + E_{\text{copy_set}} + E_{\text{red_set}} + \\ & E_{\text{source_players}} + E_{\text{global_position}} \end{aligned} \quad (3)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo MELD,
- E_{player} identifica al jugador que posee el meld,
- $E_{\text{meld_type}}$ representa el tipo de meld (chi, pon, kan, etc),
- $E_{\text{tile_set}}$ codifica las piezas que componen el meld,
- $E_{\text{copy_set}}$ distingue entre instancias duplicadas de piezas,
- $E_{\text{red_set}}$ codifica la información de dora rojo,
- $E_{\text{source_players}}$ identifica los jugadores que contribuyeron a cada pieza del meld,
- $E_{\text{global_position}}$ representa el orden cronológico de la llamada.

7.8.9 Embedding de Token de Acción

Tokens de Acciones representan eventos de acciones candidatas que son públicos y ordenados, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_{\text{action}} = & E_{\text{token_type}} + E_{\text{action_type}} + \\ & E_{\text{tile_set}} + E_{\text{copy_set}} + E_{\text{red_set}} + \\ & E_{\text{source_players}} + E_{\text{global_position}} \end{aligned} \quad (4)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo ACTION,
- $E_{\text{action_type}}$ representa descarte, riichi, chi, pon, kan, etc,
- $E_{\text{tile_set}}$ representa las piezas involucradas en la acción,
- $E_{\text{copy_set}}$ distingue entre instancias duplicadas de piezas,
- $E_{\text{red_set}}$ codifica la información de dora rojo,
- $E_{\text{source_players}}$ identifica las fuentes de las piezas para llamadas,
- $E_{\text{global_position}}$ representa el timing de la acción.

7.8.10 Embedding de Token de Estado de Juego

Tokens de Estado de Juego representan información contextual del juego que es pública y no ordenada, y su embedding es definido por la siguiente estructura:

$$E_{\text{game}} = E_{\text{token_type}} + E_{\text{round}} + E_{\text{honba}} + E_{\text{riichi_sticks}} + E_{\text{remaining_tiles}} + E_{\text{dora_set}} \quad (5)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo `GAME_STATE`,
- E_{round} codifica el viento de la ronda (este/sur),
- E_{honba} representa el número de honba actuales,
- $E_{\text{riichi_sticks}}$ codifica la cantidad de depósitos de riichi,
- $E_{\text{remaining_tiles}}$ representa las piezas restantes en la pared,
- $E_{\text{dora_set}}$ codifica los indicadores de dora actuales.

7.8.11 Embedding de Token de Estado de Jugador

El Token de Estado de Jugador representa información contextual persistente asociada con cada jugador, y su embedding es definido por la siguiente estructura:

$$E_{\text{player_state}} = E_{\text{token_type}} + E_{\text{player}} + E_{\text{seat}} + E_{\text{score}} + E_{\text{riichi_status}} + E_{\text{open_hand}} \quad (6)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo `PLAYER_STATE`,
- E_{player} codifica la identidad del jugador,
- E_{seat} representa el viento de asiento del jugador (este/sur/oeste/norte),
- E_{score} codifica los puntos actuales del jugador,
- $E_{\text{riichi_status}}$ indica si el jugador ha declarado riichi,
- $E_{\text{open_hand}}$ indica si el jugador tiene una mano abierta.

8 Resultados

En esta sección se presentarán los resultados iniciales obtenidos de un primer modelo con 100k de datos de entrenamiento y un segundo modelo con 500k de datos de entrenamiento. Se presentarán las curvas de pérdida y precisión de los modelos así como las curvas de precisión top-k. Además de eso, veremos más adelante los resultados de la policy head de los modelos, que nos permitirán ver si el modelo es capaz de aprender a predecir la probabilidad de cada tile de ser descartado en un estado de juego dado.

Para cada modelo, se presentarán las curvas de pérdida y de precisión top-1 de testeo y validación, luego se presentarán en conjunto las curvas top-3 y top-5 de validación. A continuación se presentará la distribución de confianza del modelo y finalmente los resultados de la precisión cuando separadas por categorías semánticas de error basadas en el Shanten 3.2.2 y el Ukeire 3.2.1 que será presentado en una sección posterior de comparación de modelos.

8.1 Primer Modelo

Este primer modelo fue entrenado con 100k de datos de entrenamiento, 10k de validación y 10k de testeo. El modelo fue entrenado durante 15 épocas, con un batch size de 64 y un learning rate de $1e-4$. A continuación se presentan las curvas de pérdida y precisión del modelo durante el entrenamiento y la validación.

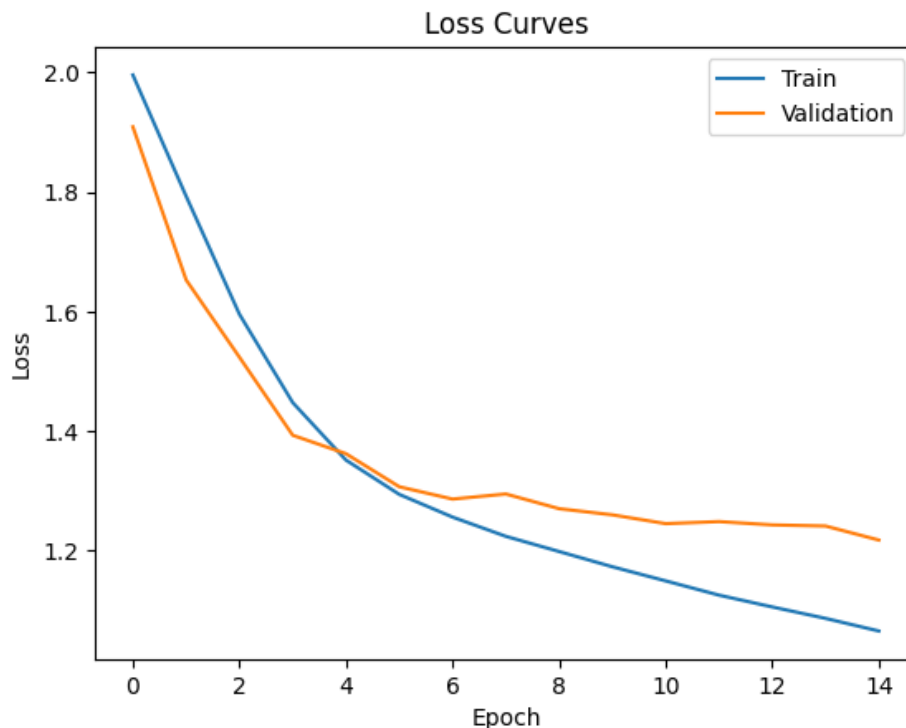


Figura 8: Curvas de pérdida y precisión del primer modelo con 100k de datos de entrenamiento.

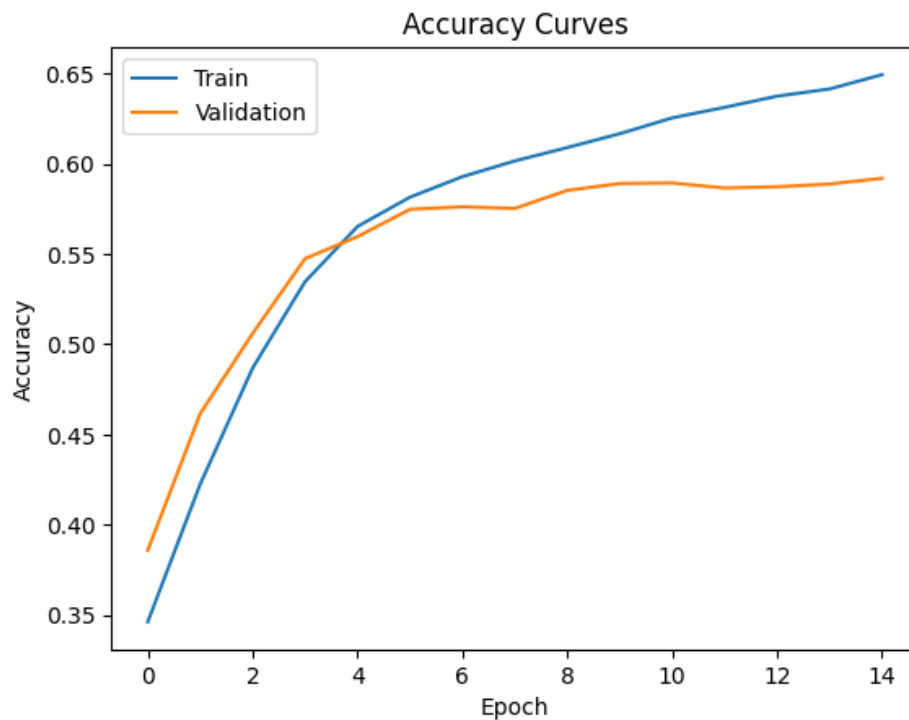


Figura 9: Curvas de pérdida y precisión del primer modelo con 100k de datos de entrenamiento.

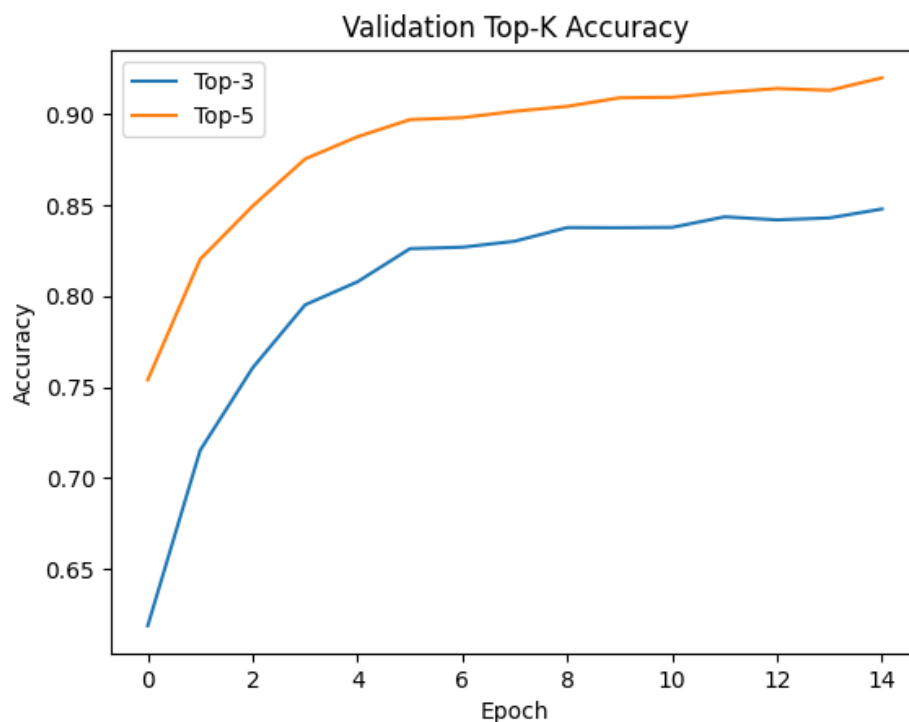


Figura 10: Curvas de precisión top-k del primer modelo con 100k de datos de entrenamiento.

Con estos resultados obtuvimos un serie de observaciones importantes: la primera es que el modelo no llegó a la saturación de la precisión, lo que nos indica que el modelo no está sobreajustando los datos de entrenamiento y que podría mejorar con más datos de

entrenamiento. La segunda observación es que apesar de que la precisión top-1 del modelo es de 0.65 para los datos de entrenamiento y 0.591 para los datos de valisación, la precisión top-3 es de 0.84 y la top-5 es de 0.91, lo que indica que el modelo es capaz de aprender a predecir los descartes de Mahjong a partir del diseño y la arquitectura que tenemos, y que con más datos de entrenamiento podría mejorar su precisión top-1.

A seguir podemos ver el gráfico de distribución de confianza del modelo, que nos permite ver la distribución de la probabilidad de los descartes predichos por el modelo.

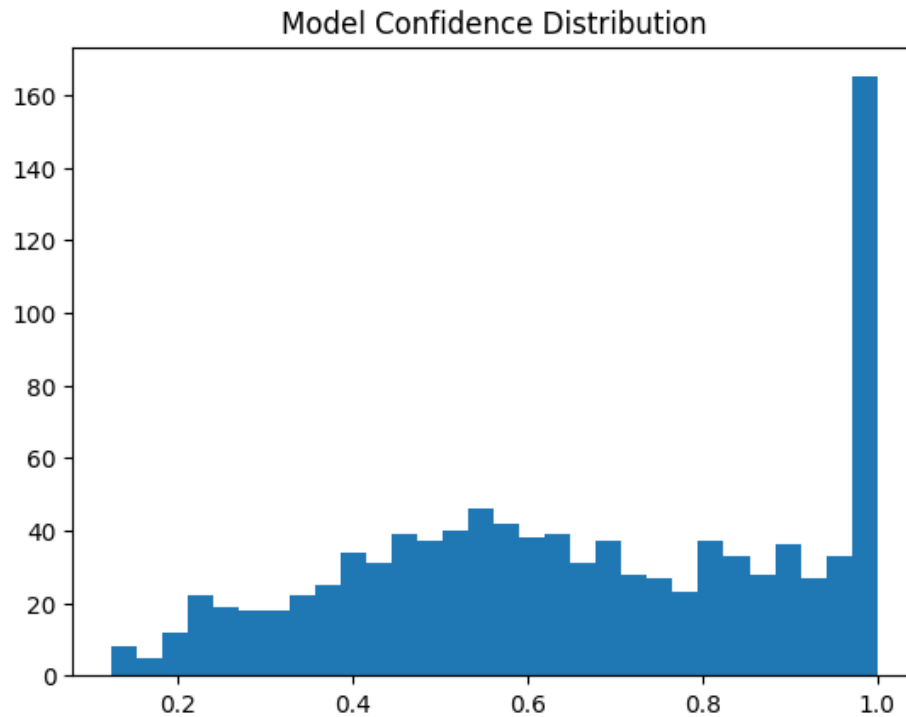


Figura 11: Distribución de confianza del primer modelo con 100k de datos de entrenamiento.

Con este gráfico podemos ver que el modelo tiene un pico en la probabilidad de 1.0. Este pico indica una confianza del 100% en muchas de las predicciones del modelo, no obstante, esta confianza puede ser explicada con los estados de juego en el que existe una única opción de descarte. Para arreglar esto, se realizó una limpieza de datos para eliminar los estados de juego en los que existe una única opción de descarte para el entrenamiento del próximo modelo, que será presentado a continuación.

8.2 Segundo Modelo

Este segundo modelo fue entrenado con 500k de datos de entrenamiento, 50k de validación y 50k de testeo. El modelo fue entrenado durante 15 épocas, con un batch size de 64 y un learning rate de $1e-4$. A continuación se presentan las curvas de pérdida y precisión del modelo durante el entrenamiento de las épocas 9 a 15. El dataset de entrenamiento también fue limpiado para eliminar los estados de juego en los que existe una única opción de descarte.

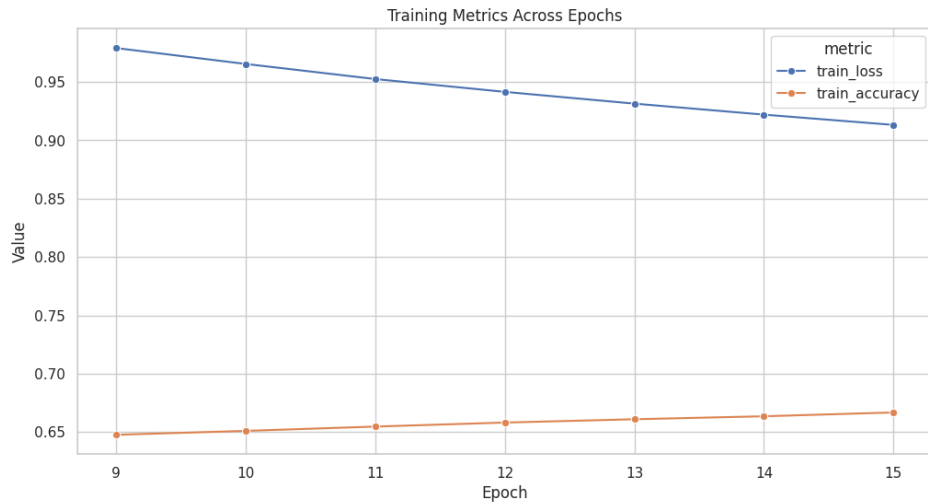


Figura 12: Curvas de pérdida y precisión del segundo modelo con 500k de datos de entrenamiento.

Y en esta figura podemos ver el gráfico de distribución de confianza del modelo ajustado sin los estados de juego en los que existe una única opción de descarte.



Figura 13: Distribución de confianza del segundo modelo con 500k de datos de entrenamiento.

Podemos ver que pese a la disminución de la confianza del modelo, el modelo sigue teniendo un pico en la probabilidad de 1.0, lo que indica que el modelo sigue teniendo una confianza del 100% en muchas de sus predicciones. Además de eso, podemos ver que la tiene un segundo pico en el 0.5 donde se mantiene estable hasta el 0.9. Estos resultados indican una buena confianza del modelo en sus predicciones, principalmente si consideramos que en su mayoría de los estados de juego existen 13 opciones de descarte posibles en media.

8.3 Validación Estratégica

La validación estratégica es un proceso que nos permite evaluar la capacidad del modelo para predecir descartes de Mahjong en función de la estrategia de juego. Para ello, se utilizaron métricas basadas en el Shanten y el Ukeire, que nos permiten clasificar los errores del modelo en diferentes categorías semánticas.

Las categorías creadas son las siguientes:

- Exact match: el descarte predicho por el modelo coincide exactamente con el descarte real del jugador.
- Equivalent: el descarte predicho por el modelo resulta en el mismo Shanten y Ukeire que el descarte real del jugador.
- Better shanten: el descarte predicho por el modelo resulta en un Shanten menor que el descarte real del jugador.
- Better ukeire: el descarte predicho por el modelo resulta en un Ukeire mayor que el descarte real del jugador.
- Near equivalent: el descarte predicho por el modelo resulta en un Shanten y Ukeire que difieren en máximo dos unidades del descarte real del jugador.
- Shanten loss: el descarte predicho por el modelo resulta en un Shanten mayor que el descarte real del jugador.
- Ukeire loss: el descarte predicho por el modelo resulta en un Ukeire menor.

8.3.1 Validación Estratégica del Primer Modelo

A partir de 10k de datos de validación, de los cuales todos tienen por lo menos más de una opción de acción válida, se realizó la categorización de cada decisión tomada por la máquina utilizando las categorías mencionadas anteriormente.

En este primer gráfico podemos observar que muchas decisiones buenas, como *better_shanten* o *better_ukeire* no fueron llevadas en consideración al *accuracy* del modelo:

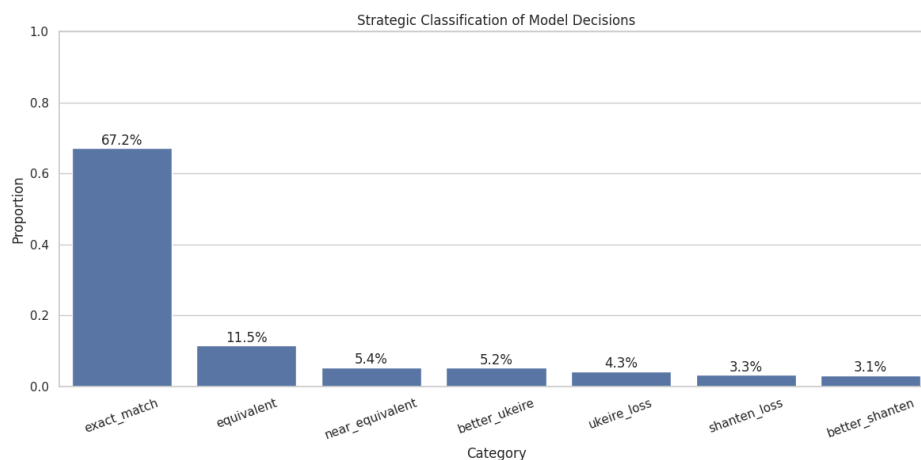


Figura 14: Porcentaje de Cada Categoría Estratégica en el Primer Modelo

A partir de esto, podemos sacar un valor de *strategic accuracy* mucho mejor que el *accuracy* original, sumando las categorías de *exact_match*, *equivalent*, *better_ukeire*, *better_shanten* obtenemos que el modelo en realidad obtuvo una *accuracy* de 92.4%.

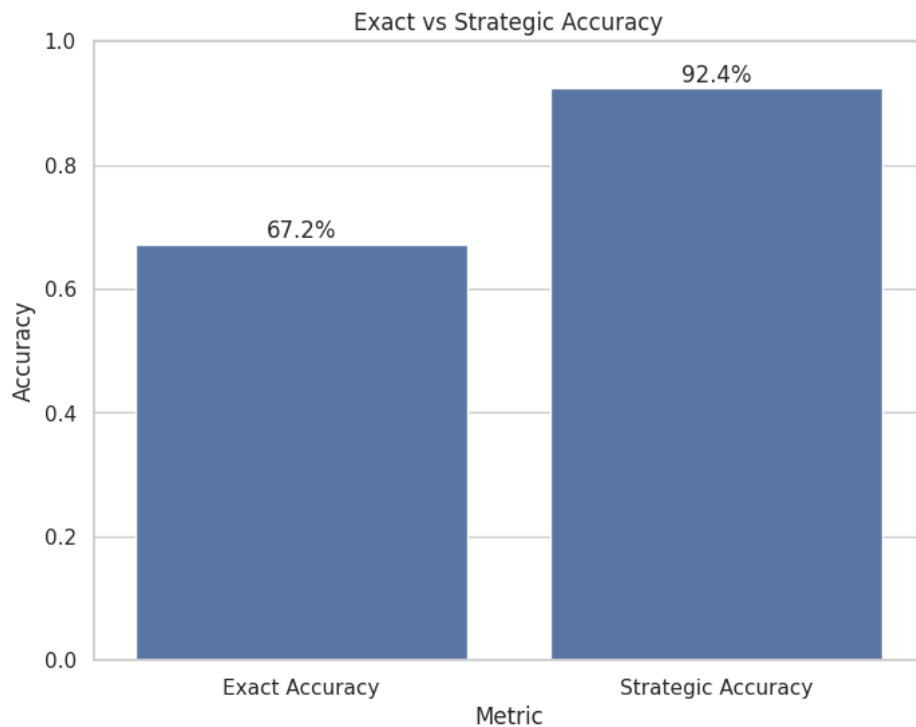


Figura 15: Comparación validación de accuracy exacta vs accuracy estratégica en el Primer Modelo

En esta figura también podemos ver la confianza del modelo sobre estos datos sin la interferencia de la confianza para datos de validación con una sola acción válida:

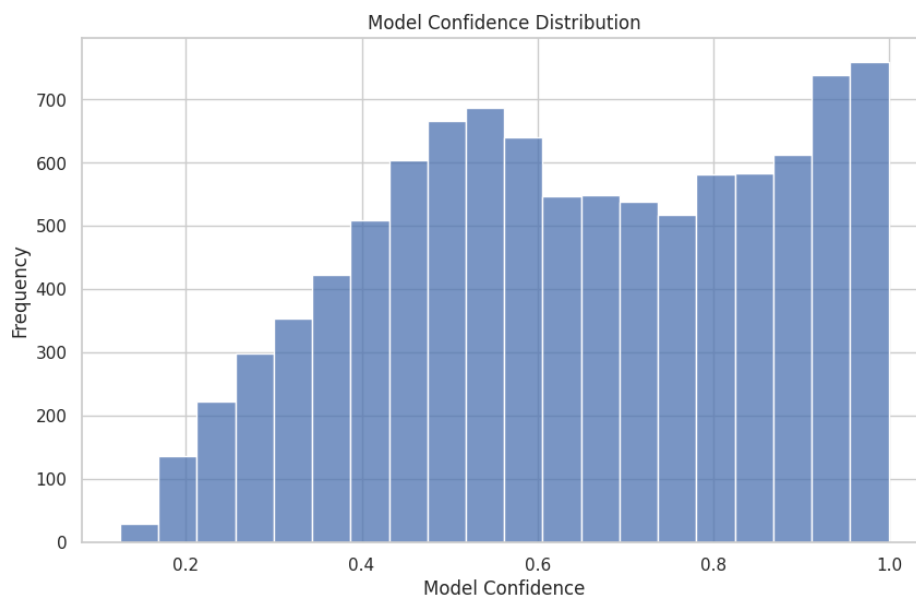


Figura 16: Distribución de confianza del primer modelo con ajustado al dataset sin opciones unitarias

Como podemos ver, la confianza del modelo está en rangos apropiados para el número de opciones que tiene normalmente (una media de 13 opciones).

Por último, podemos ver también la distribución de confianza del modelo por categoría estratégica, y se percibe que para los casos en que hubo errores graves la confianza del modelo es muy baja en comparación a la confianza en casos de exactitud del modelo, esto indica un buen aprendizaje del modelo.

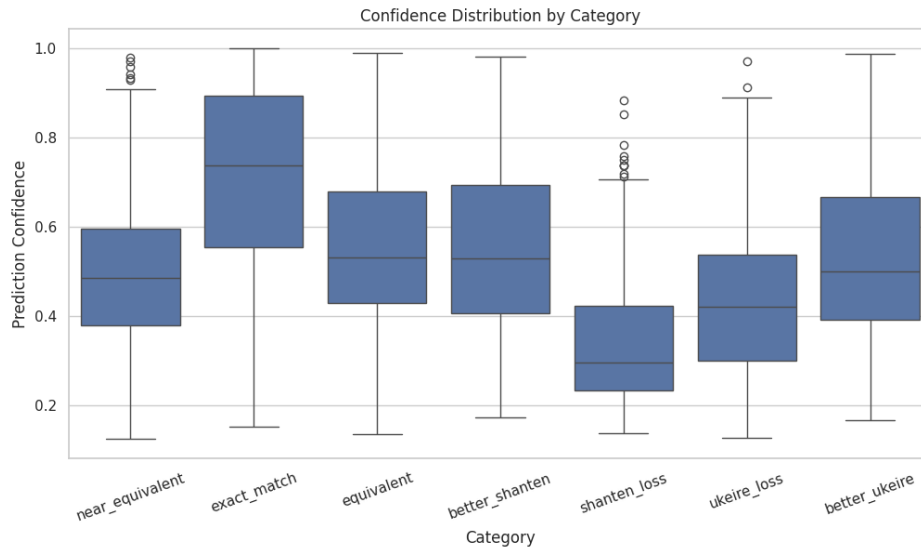


Figura 17: Distribución de confianza del primer modelo por categoría de la validación estratégica

8.3.2 Validación Estratégica del Segundo Modelo

De manera análoga al análisis previo, se evaluaron 10k muestras de validación con múltiples opciones de acción válidas para el segundo modelo, aplicando la misma metodología de categorización estratégica. Al inspeccionar la distribución de las decisiones, observamos cómo el aumento de datos impactan en la consideración de jugadas estratégicamente superiores como *better_shanten* o *better_ukeire*:

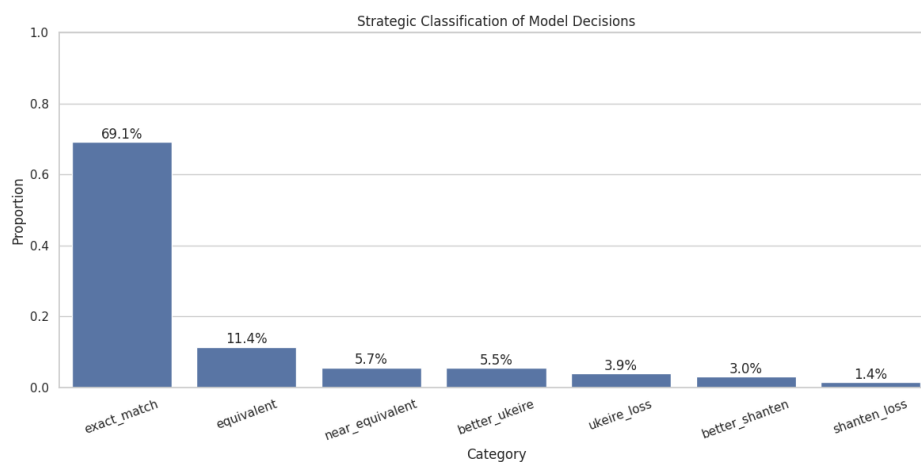


Figura 18: Porcentaje de Cada Categoría Estratégica en el Segundo Modelo

Calculando nuevamente el *strategic accuracy* acumulando las categorías de *exact_match*,

equivalent, *better_ukeire* y *better_shanten*, obtenemos un valor ajustado de **94.6%**. Esto representa una mejora significativa respecto al accuracy crudo, demostrando que el modelo captura mejor la esencia estratégica del juego más allá de la coincidencia exacta:

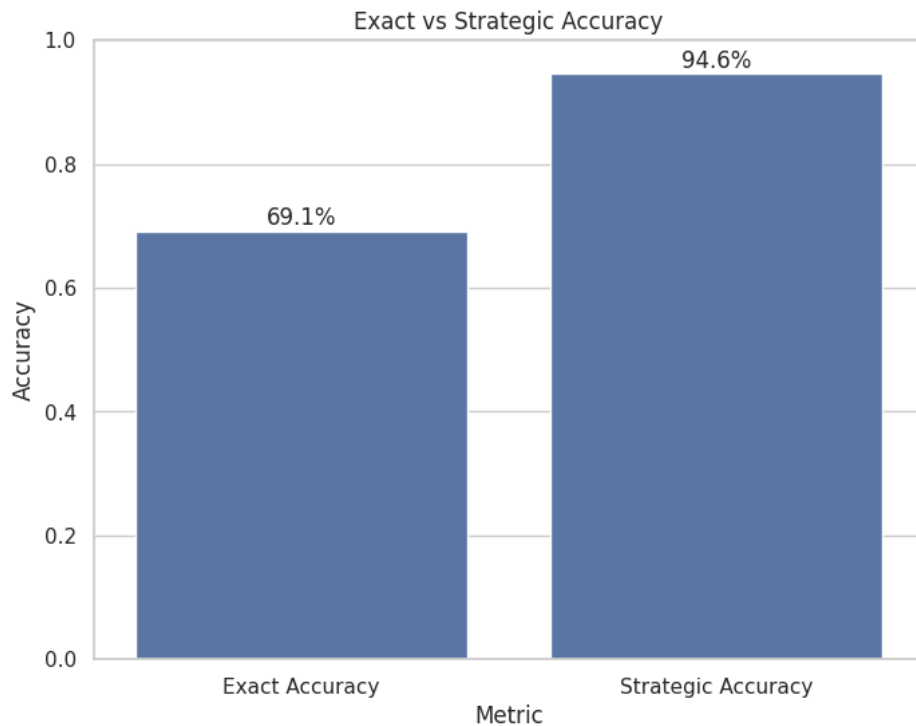


Figura 19: Comparación validación de accuracy exacta vs accuracy estratégica en el Segundo Modelo

Respecto a la confianza del modelo, la distribución reflejada es bastante similar a la del primer modelo, pero aumenta considerablemente la confianza en 100%:

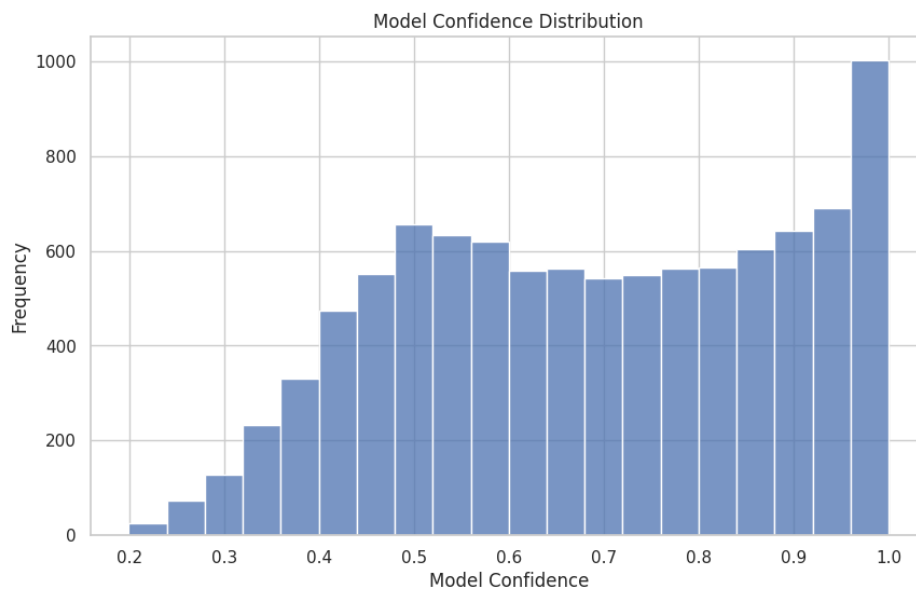


Figura 20: Distribución de confianza del segundo modelo

Finalmente, podemos ver la descomposición de la confianza por categoría estratégica en este segundo modelo, donde más adelante se realizará una comparación entre los dos modelos:

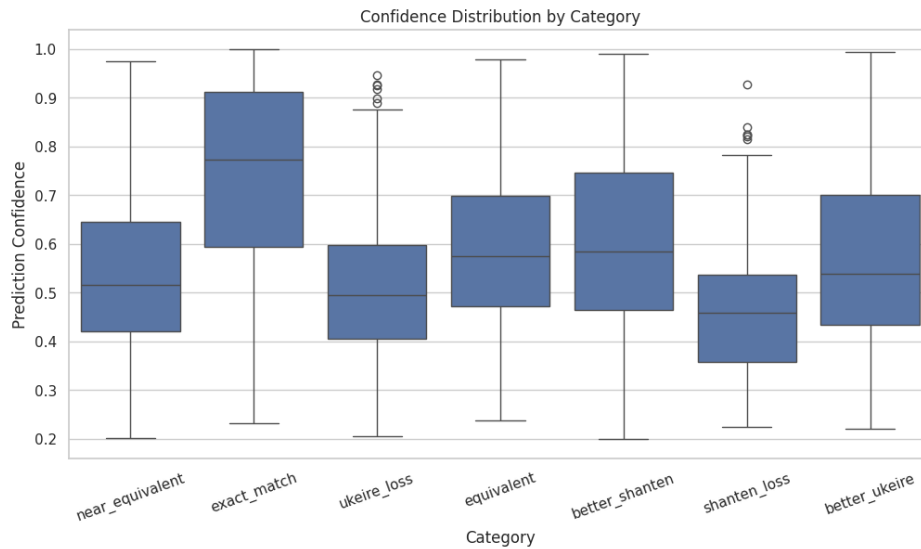


Figura 21: Distribución de confianza del segundo modelo por categoría de la validación estratégica

8.3.3 Análisis Comparativo: Impacto del Volumen de Datos

Para evaluar el impacto directo de la cantidad de datos en el rendimiento estratégico y exacto, se realizó una comparación entre el modelo entrenado con 100k muestras y aquel entrenado con 500k muestras.

En primer lugar, observamos que el aumento del conjunto de entrenamiento generó una mejora significativa en la *exact action accuracy*. El modelo con 500k datos alcanzó un valor de 0.691, superando al modelo con 100k, cuyo rendimiento se situó en 0.671. Esta tendencia positiva también se reflejó en la métrica ajustada: el *strategic accuracy* aumentó del 92.4% al 94.6%, lo que indica una mayor capacidad del modelo para identificar acciones equivalentes o superiores desde una perspectiva estratégica general:

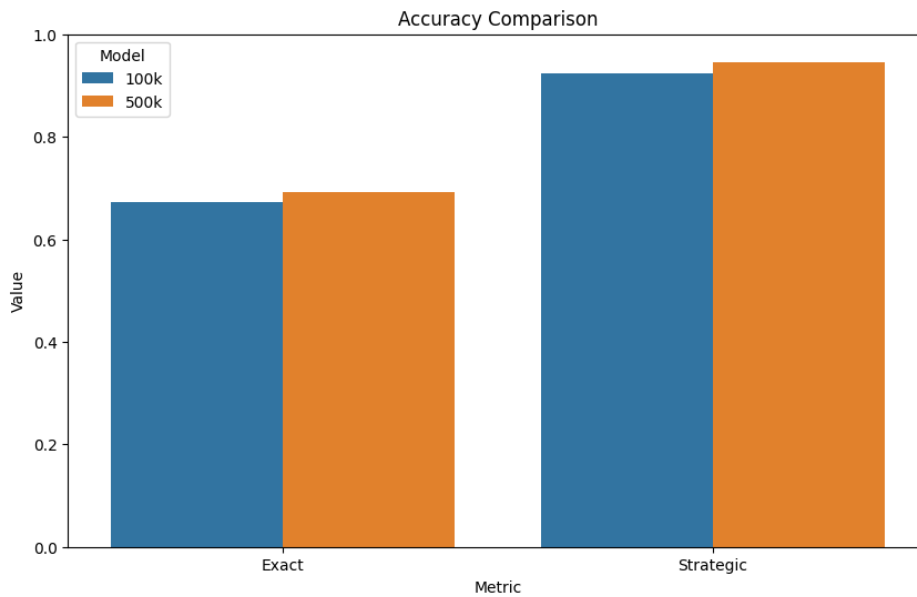


Figura 22: Comparación de Accuracy Exacta y Estratégica entre modelos con 100k y 500k de datos

Al analizar la distribución específica por categorías estratégicas, resulta evidente un cambio cualitativo crucial en la toma de decisiones. El modelo entrenado con el conjunto más amplio redujo drásticamente la frecuencia de decisiones que degradan el estado del juego (*shanten-degrading decisions*), pasando de un 3.4% a solo un 1.8%. Esta reducción sugiere que el modelo no solo está memorizando respuestas exactas, sino aprendiendo patrones estructurales más profundos que evitan errores estratégicos graves:

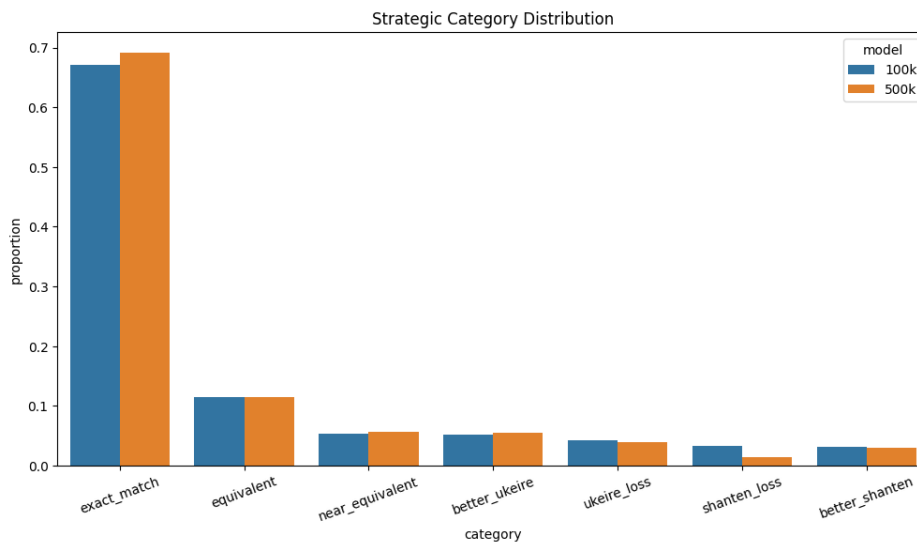


Figura 23: Comparación de la distribución de categorías estratégicas entre los dos modelos

En conclusión, los resultados muestran que el incremento del conjunto de entrenamiento de 100k a 500k muestras produce una mejora consistente en prácticamente todas las métricas evaluadas. El segundo modelo aumenta tanto la *exact accuracy* (67.16% frente a 69.13%) como la *strategic accuracy* (92.43% frente a 94.62%), además de presentar una mayor confianza media en sus predicciones. Desde el punto de vista estratégico, también se observa

una reducción importante de las decisiones que empeoran el Shanten y una mejora del *Mean Ukeire Difference*, que pasa de un valor negativo a uno positivo. Aunque el porcentaje de decisiones clasificadas como *Ukeire Loss* aumenta muy ligeramente (4.3% frente a 4.4%), este incremento es prácticamente despreciable y viene acompañado de un mayor número de decisiones *Better Ukeire*, por lo que el balance global continúa siendo claramente favorable para el modelo entrenado con 500k muestras. Estos resultados indican que el aumento del volumen de datos no solo mejora la capacidad del modelo para reproducir decisiones expertas, sino también para realizar descartes estratégicamente más sólidos en una mayor variedad de situaciones de juego.

9 Conclusiones y Líneas de Trabajo Futuro

El objetivo principal de este trabajo ha sido estudiar la aplicación de arquitecturas Transformer al problema de la predicción de descartes en Riichi Mahjong mediante aprendizaje por imitación utilizando partidas reales de Tenhou. Para ello se diseñó una representación propia del estado de juego, un proceso de tokenización adaptado al dominio y un modelo basado en un Transformer Encoder capaz de aprender a seleccionar descartes a partir de ejemplos de jugadores expertos.

Los resultados obtenidos muestran que el modelo es capaz de aprender patrones estratégicos relevantes a partir de los datos disponibles. El incremento del conjunto de entrenamiento desde 100k hasta 500k muestras produjo mejoras tanto en la precisión exacta como en la precisión estratégica, además de reducir significativamente el número de decisiones que empeoran el Shanten y mejorar la diferencia media de Ukeire. Estos resultados indican que el modelo no únicamente memoriza ejemplos concretos, sino que comienza a capturar relaciones estratégicas presentes en las partidas de entrenamiento.

No obstante, la principal aportación de este trabajo no reside únicamente en el desarrollo del modelo, sino en la metodología propuesta para su evaluación. Tradicionalmente, los modelos de aprendizaje por imitación suelen evaluarse mediante métricas de clasificación, como la exactitud (*accuracy*), que consideran correcta únicamente la reproducción exacta de la acción realizada por el jugador experto. En un juego como el Riichi Mahjong, donde una misma situación puede admitir múltiples decisiones estratégicamente equivalentes, este criterio resulta insuficiente y puede conducir a conclusiones erróneas sobre la calidad real del modelo.

Con el objetivo de superar esta limitación, en este trabajo se propuso una validación estratégica basada en conceptos propios del Mahjong, como el Shanten y el Ukeire. Esta metodología permite distinguir entre errores realmente perjudiciales y decisiones alternativas que mantienen o incluso mejoran la calidad estratégica de la jugada. Gracias a este enfoque fue posible observar que una parte importante de las predicciones consideradas incorrectas por las métricas tradicionales corresponden, en realidad, a decisiones estratégicamente válidas o incluso superiores a las realizadas por el jugador experto.

Sin embargo, estas métricas no deben interpretarse como una medida absoluta de la inteligencia del modelo. Una precisión estratégica elevada no implica necesariamente que el sistema haya comprendido todos los aspectos del juego, sino únicamente aquellos representados por las heurísticas utilizadas durante la evaluación. En consecuencia, estas métricas deben entenderse como una herramienta de análisis que permite describir con mayor precisión el comportamiento del modelo, y no como una medida definitiva de su nivel de juego.

10 Líneas de Trabajo Futuro

Aunque los resultados obtenidos son prometedores, existen numerosas líneas de trabajo que permitirían ampliar y mejorar tanto el modelo desarrollado como la metodología de evaluación propuesta.

En primer lugar, una posible línea de investigación consiste en ampliar las métricas estratégicas utilizadas durante la evaluación. En este trabajo únicamente se consideraron el Shanten y el Ukeire como indicadores de calidad estratégica. Sin embargo, el Riichi Mahjong incorpora numerosas heurísticas adicionales relacionadas con el juego ofensivo y defensivo, como el Suji, Ura-Suji, Kabe, Genbutsu o el análisis del peligro de descarte frente a jugadores en Riichi. La incorporación de este tipo de criterios permitiría construir una evaluación mucho más completa del comportamiento estratégico del modelo.

De forma similar, la propia clasificación estratégica podría extenderse para analizar otros aspectos de la toma de decisiones, permitiendo estudiar no solamente si un descarte mejora la eficiencia de la mano, sino también si representa una decisión defensivamente adecuada o consistente con estrategias de mayor nivel empleadas por jugadores expertos.

Desde el punto de vista del modelo, otra línea de trabajo consiste en explorar arquitecturas más avanzadas. Aunque en este proyecto se ha empleado un Transformer Encoder relativamente sencillo para facilitar el análisis del comportamiento del sistema, existen variantes modernas como Decision Transformer o arquitecturas específicas para aprendizaje secuencial que podrían aprovechar mejor la naturaleza temporal de las partidas de Mahjong.

Asimismo, este trabajo se ha centrado exclusivamente en la decisión de descarte, que constituye la acción más frecuente durante una partida. Como continuación natural del proyecto, sería interesante ampliar el espacio de acciones para incluir decisiones como Chi, Pon, Kan, Riichi o incluso Tsumo y Ron. Esto permitiría construir un agente capaz de participar de manera completa en una partida real de Riichi Mahjong.

Finalmente, otra línea especialmente interesante consiste en utilizar el modelo obtenido mediante aprendizaje por imitación como punto de partida para un proceso posterior de aprendizaje por refuerzo. En este contexto, las métricas estratégicas desarrolladas en este trabajo podrían resultar útiles para el diseño de funciones de recompensa más informativas que la simple victoria o derrota al final de la partida. Incorporar información relacionada con el Shanten, el Ukeire o futuras heurísticas estratégicas permitiría proporcionar una señal de aprendizaje más densa durante el entrenamiento, facilitando potencialmente la convergencia del agente y favoreciendo la adquisición de comportamientos estratégicamente sólidos.

Referencias

- [1] D. Chiba, *Riichi Book 1: The Ultimate Guide to the Japanese Game Taking the World by Storm*. Lulu.com, 2015, ISBN: 978-1329626478.
- [2] T. Developers, *Tjong AI Mahjong Platform*, <https://github.com/tjong>, Open-source Riichi Mahjong AI platform, 2024.
- [3] J. Devlin, M. Chang, K. Lee y K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *CoRR*, vol. abs/1810.04805, 2018. arXiv: 1810.04805. dirección: <http://arxiv.org/abs/1810.04805>
- [4] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. MIT Press, 2016. dirección: <https://www.deeplearningbook.org/>
- [5] A. Hussein et al., “Imitation Learning: A Survey of Learning Methods”, *ACM Computing Surveys*, 2017.
- [6] Japanese Mahjong Wiki. “Rules overview”, visitado 9 de jun. de 2026. dirección: https://riichi.wiki/Rules_overview
- [7] J. Li et al., “Suphx: Mastering Mahjong with Deep Reinforcement Learning”, *arXiv preprint arXiv:2003.13590*, 2020.
- [8] A. Vaswani et al., “Attention Is All You Need”, *NeurIPS*, 2017.